TECHNISCHE UNIVERSITÄT DRESDEN

KAAI:
Knowledge-aware
Artificial Intelligence

Faculty of Computer Science

Institute for Artificial Intelligence
Chair of Knowledge-aware Artificial Intelligence

Master Thesis

Hybrid Representation and Retrieval of Historical Battles Using Narrative Structures and Embeddings

Vladyslav Romanov

Born on: 14.03.2001 in Lugansk
Matriculation number: 5127711

November 3, 2026

First referee
Prof. Dr. Simon Razniewski

Second referee
Prof. Dr.-Ing. Michael Färber

Supervisor
Prof. Dr. Simon Razniewski

Statement of authorship

I hereby certify that I have authored this document entitled *Hybrid Representation and Retrieval of Historical Battles Using Narrative Structures and Embeddings* independently and without undue assistance from third parties. No other than the resources and references indicated in this document have been used. I have marked both literal and accordingly adopted quotations as such. There were no additional persons involved in the intellectual preparation of the present document. I am aware that violations of this declaration may lead to subsequent withdrawal of the academic degree.

Dresden, November 3, 2026

Vladyslav Romanov

Abstract

Large Language Models (LLMs) have been increasingly studied for knowledge-intensive tasks across different domains such as finance, medicine, law, scientific discovery and open-domain question answering. Retrieval-augmented generation (RAG) techniques can improve generation quality and reduce hallucinations by providing external knowledge to LLMs. However, both conventional RAG and general LLM querying may be less effective for complex domains containing narrative-rich information, as well as for database-style queries such as SELECT, COUNT, and ORDER BY. The main goal of this thesis is to investigate whether providing LLMs with structured data containing narrative information can improve answer quality and reduce hallucinations in the domain of historical battles. First, we construct a large-scale knowledge base of 11,561 historical battles identified via Wikidata. Each battle is represented in a structured JSON file containing metadata, military facets, narrative information and patterns, generated from the raw text of the corresponding Wikipedia article using the GPT-OSS-120B model. The structured battle representations are then retrieved using three different retrieval approaches and provided as context to the same model. During the experimental part, we compare the question-answering performance of the baseline LLM and context-augmented LLM using several evaluation metrics. The main contribution of this thesis is to design a structured representation and retrieval framework for the historical battles domain and analyze how structured context affects LLM-based question-answering performance compared with a baseline LLM.

Contents

1 Introduction 5
- 1.1 Motivation . . . 5
- 1.2 Problem Statement . . . 9
- 1.3 Research Questions . . . 10
- 1.4 Our Approach . . . 10
- 1.5 Contributions . . . 11

2 Related work 13
- 2.1 Narrative Representations . . . 13
- 2.2 Schema-guided event extraction . . . 14
- 2.3 Retrieval Methods and RAG . . . 14
- 2.4 Battle-specific Retrieval . . . 15

3 Methodology 17
- 3.1 Data Sources and Knowledge Base Construction . . . 17
- 3.2 Battle Representation Schema . . . 18
- 3.3 Wikidata and JSON Schema Correspondence and Coverage . . . 23
- 3.4 Factual Validation of the JSON Knowledge Base . . . 23
- 3.5 LLM-Based Schema Population . . . 24
- 3.6 LLM Verification and Regeneration . . . 27
- 3.7 Retrieval Methods . . . 29
  - 3.7.1 Lexical Retrieval with BM25 . . . 30
  - 3.7.2 Dense Retrieval . . . 32
  - 3.7.3 Hybrid Retrieval . . . 34
- 3.8 Answer Generation . . . 34
  - 3.8.1 Oracle-context generation . . . 35
  - 3.8.2 End-to-end generation . . . 37

4 Experimental Design 39
- 4.1 Query Set . . . 39
- 4.2 Oracle-context Generation Experiment . . . 41
- 4.3 Retrieval Experiment . . . 42
- 4.4 End-to-end RAG Experiment . . . 45

5 Experimental Results 47
- 5.1 Retrieval Results . . . 47
  - 5.1.1 Overall Retrieval Performance . . . 47

5.1.2 Retrieval Performance for Common and Rare Battles 49
5.1.3 Retrieval Performance by Query Type 51
5.2 Oracle-context Generation Results 53
5.2.1 Overall Generation Performance 53
5.2.2 Generation Performance for Common and Rare Battles 55
5.3 End-to-End RAG Evaluation Results 57
5.3.1 Overall Performance and Error Analysis 57
5.3.2 Faithfulness and Context Relevance 59
5.3.3 End-to-end RAG Performance for Common and Rare Battles 61
5.3.4 End-to-end RAG Performance by Query Type 62
5.4 JSON Knowledge Base Validation Results 65

6 Discussion 67
6.1 Key Findings and Conclusions 67
6.1.1 Research Objective 67
6.1.2 Answers to the Research Questions 67
6.1.3 Overall Conclusion 68
6.2 Limitations and Future Work 68
6.2.1 Benchmark Grounding and Historical Validity 68
6.2.2 Wikipedia Data Quality 69
6.2.3 Limited Knowledge Base Coverage 69
6.2.4 Field-Aware Lexical Retrieval 70
6.2.5 Schema Normalization 71
6.2.6 Query Set Construction 72

1 Introduction

1.1 Motivation

Large Language Models (LLMs) have been widely studied and applied to question-answering tasks across different domains, including finance (Sarmah et al., 2024), medicine (Wang and Zhang, 2024), law (Lai et al., 2024), and scientific discovery (Zhang et al., 2024). For knowledge-intensive tasks, Retrieval-Augmented Generation (RAG) is particularly useful because generation is supported by external evidence rather than relying solely on the model's internal knowledge (Lewis et al., 2020). However, conventional RAG may be less effective for narrative-rich texts, because it tends to process information at the level of isolated documents or passages and can therefore overlook underlying event structures, temporal development and logical relationships between events (Yang et al., 2025). Moreover, classical RAG is also limited for exhaustive and database-style queries such as *find all battles satisfying condition X*. Such queries would require the RAG system to check this condition across the entire collection of documents, whereas RAG typically retrieves only a small top-k subset of them. These limitations are particularly relevant to domains in which information is organized as complex narratives rather than isolated facts. Movies provide one such example, where adapting LLMs for narrative understanding remains an underexplored challenge (Shen and Garg, 2025). Historical battles represent another domain with rich event and narrative structure. A recent study has shown that simple quantitative statistics alone may fail to capture higher-level tactical patterns and similarities between battles (Coulson et al., 2023). Instead, meaningful similarities can emerge from the sequences and relationships between battle actions. This suggests that historical battles may benefit from richer representations that capture not only factual attributes but also the development and sequence of events.

In practice, many proprietary and open-weight models indeed struggle with database-style questions in complex narrative domains such as historical battles. As an example, we ask two well-known proprietary models, Gemini (Gemini Team, 2023) and Claude (Anthropic, 2026a), several battle-related questions. It is especially interesting to see how these models are able to capture narrative-rich facts about less-known battles. First, we ask a model a question and prohibit any web search. If the model fails to mention relevant battles, we provide the gold labels and ask whether the model recognizes its mistake. Figure 1.1 and Figure 1.2 show the question asked to Gemini 3.6 Flash (Google, 2026). In this particular example, the model fails to mention two relevant battles, and after allowing it to use web search and explicitly asking about the missing battles, the model recognizes its mistake. Figures 1.3

and 1.4 demonstrate an analogous example with Claude Sonnet 5.5 (Anthropic, 2026b). The situation is similar: the model fails to mention two gold battles, but after enabling web search, it recognizes its mistake. For completeness, we also performed the same test with the open-weight reasoning model GPT-OSS-120B, as shown in Figure 1.5. Similarly, the model was not able to mention two gold battles. Both queries used for these models have two things in common. First, they have a database-style nature, and second, the relevant battles are relatively obscure and may therefore have been underrepresented in the models' training corpora. Listings 1.1 and 1.2 demonstrate how these queries can be represented as SQL-style queries, further illustrating their database-style nature.

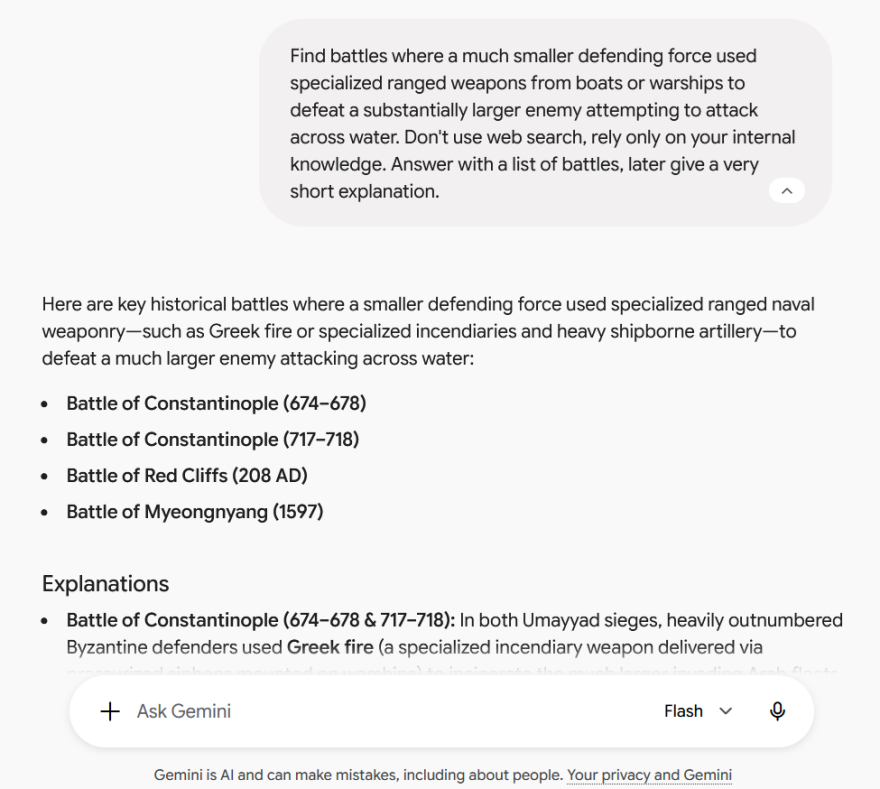


Figure 1.1: Initial Gemini response to a complex historical battle query.

```sql
SELECT battle
FROM battles
WHERE defender_strength / attacker_strength < 0.5
  AND defender_weapon_type = 'specialized ranged weapon'
  AND defender_platform IN ('boat', 'warship')
  AND attacker_terrain = 'water crossing'
  AND winner = 'defender';
```

Listing 1.1: SQL-style representation of a historical battle query involving specialized naval weapons and numerical superiority.

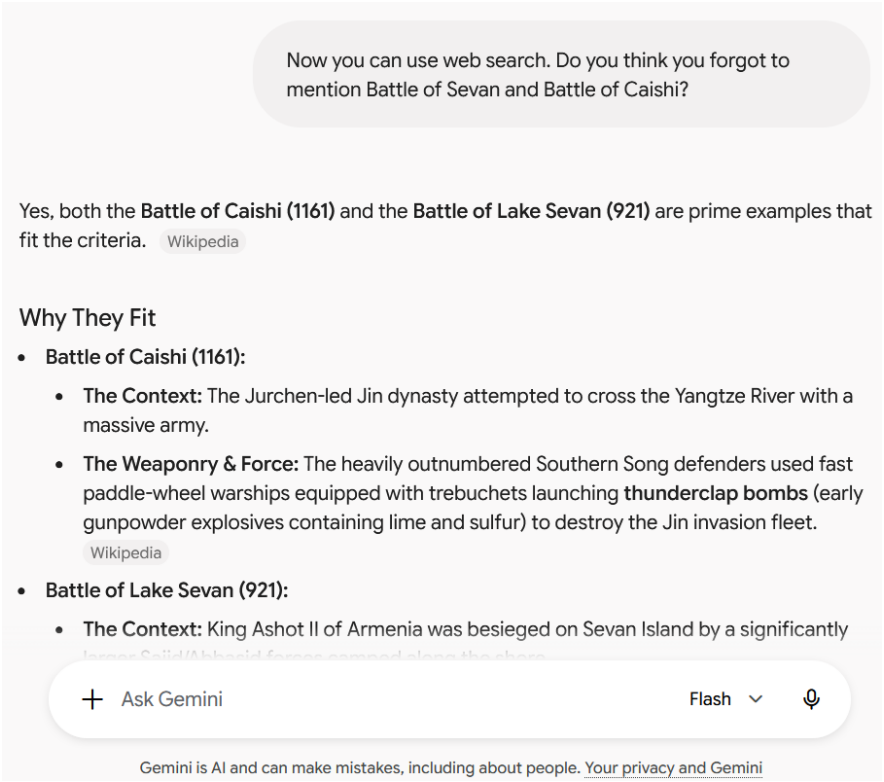


Figure 1.2: Gemini response to a follow-up query, acknowledging the omission of the Battles of Caishi and Lake Sevan from the initial answer.



Figure 1.3: Initial Claude response to a complex historical battle query.

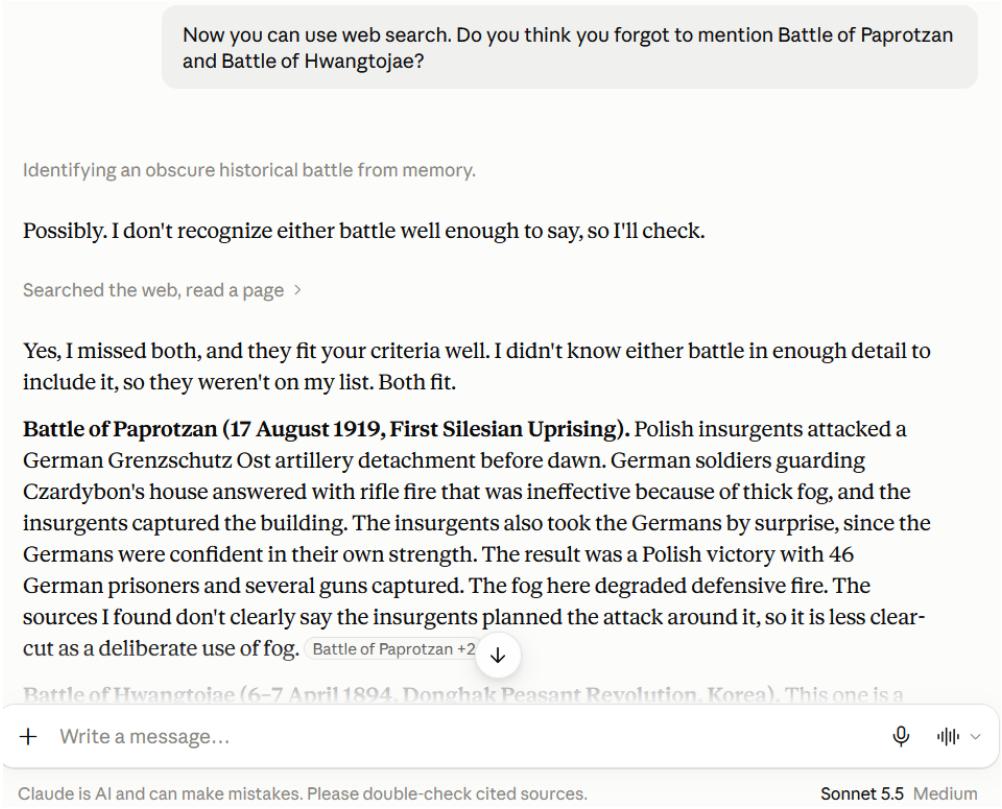


Figure 1.4: Claude response to a follow-up query, acknowledging the omission of the Battles of Paprotzan and Hwangtojae after using web search.

```
(hybridb_env) PS C:\Users\vlad8\Projects-VS-Code\HybridB> python .\ask_llm.py
Enter your question: Find battles where dense fog was deliberately exploited to conceal an approach, surprise the enemy, or mak
e defensive fire less effective during a successful attack. Answer with a list of battles without details, later give a very sh
ort explanation.

Answer:
**Battles (list only):**
- Battle of the Bulge (Ardennes Offensive, December 1944)
- Battle of Hürtgen Forest (September – December 1944)
- Battle of the Lys (April 1918)
- Battle of Antietam (September 17 1862)
- Battle of the Somme (July 1916) – early-morning fog phase
- Battle of the Seelow Heights (April 1945)
- Battle of the Smolensk (July 1941) – fog-assisted German advance

**Very short explanations:**
- **Ardennes (Bulge):** Thick fog concealed the German surprise attack, limiting Allied air reconnaissance and artillery spotti
ng.
- **Hürtgen Forest:** Persistent fog reduced visibility, allowing German defenders to ambush advancing U.S. troops and blunt Al
lied fire.
```

Figure 1.5: GPT-OSS-120B response to a historical battle query involving the tactical use of fog, generated using the model's internal knowledge without external retrieval.

```sql
SELECT battle
FROM battles
WHERE weather_condition = 'dense fog'
  AND fog_exploitation = 'deliberate'
  AND tactical_effect IN (
      'conceal approach',
      'surprise enemy',
      'reduce defensive fire effectiveness'
  )
  AND attack_outcome = 'successful';
```

Listing 1.2: SQL-style representation of a historical battle query involving the deliberate tactical exploitation of fog.

The aim of this thesis is to investigate whether structured battle representations can improve the factual accuracy and overall quality of LLM-generated answers in the domain of historical battles. For this purpose, we first design a structured battle representation schema and construct a local knowledge base containing structured battle representations. We then analyze different retrieval strategies and compare different context-augmented model configurations.

1.2 Problem Statement

Recent studies on event-centric knowledge representation motivate structured representations for narrative-rich knowledge (Guan et al., 2023). Research on narrative understanding further highlights the importance of capturing information beyond isolated facts, including temporal and event structure (Hamilton et al., 2026). However, it remains unclear whether rich structured representations can substantially improve the accuracy of answers, particularly in the domain of historical battles. Our work investigates whether an LLM provided with additional structured context can produce more accurate and better-grounded answers than the same model relying only on its internal knowledge or on raw textual battle passages. By design, when we evaluate the augmented models, we instruct them not to use their internal knowledge. This makes it easier to determine whether a correct answer is supported by the provided context rather than by the model's parametric memory. By explicitly requiring the model to rely only on the supplied battle records, the experiment evaluates whether the structured knowledge base contains sufficient information to answer the questions and whether the model can successfully extract and reason over that information. This also enables a more controlled comparison with raw-text augmentation, where the model receives information from the same underlying sources, but in the form of raw-text Wikipedia passages.

This problem becomes more challenging at scale. In this work, we consider 11,561 battle instances identified through Wikidata (Vrandečić and Krötzsch, 2014), so relevant battle representations must first be retrieved from a large candidate knowledge base before they can be provided to the LLM. This makes retrieval quality another central challenge for this thesis. Prior work on dense retrieval has highlighted the importance of retrieval quality for downstream question-answering performance in open-domain QA (Karpukhin et al., 2020). This study evaluates both top-k retrieval accuracy and end-to-end QA performance, illustrating the connection between the retrieved context and the final answer. Following a similar evaluation approach, we investigate three different retrieval strategies and compare their ability to identify relevant battles from all 11,561 instances. This allows us to determine which retrieval strategy is the most effective for our task.

1.3 Research Questions

In this thesis, we consider three main research questions, referred to as RQ1–RQ3.

RQ1. How can historical battles be represented through a structured schema that captures factual, military, temporal, spatial, participant-related, outcome-related, and narrative information?

RQ2. How do lexical, dense, and hybrid retrieval methods perform when retrieving relevant battle representations from a large structured battle knowledge base?

RQ3. How does the use of structured battle representations as external evidence affect LLM-based question-answering quality compared with direct LLM querying and raw-text augmentation?

1.4 Our Approach

To address these research questions, we first define a domain-specific representation schema for historical battles. At the conceptual level, each battle is represented as an event containing quantitative and factual properties, such as date, location, participants' strength, and losses, as well as military-domain attributes and narrative information, including event order, turning points, narrative patterns, and consequences. Based on this conceptual representation, we design a JSON schema that organizes this information into predefined fields.

The next step is to populate this representation at scale. We identify historical battle instances through Wikidata and retrieve the raw text of their corresponding Wikipedia articles using the Wikipedia API. We then use an LLM to process the collected articles and populate the predefined JSON schema for each battle instance. Following this process, we obtain a structured knowledge base of historical battle representations.

As the next important step, we investigate three retrieval strategies for selecting relevant battles from the knowledge base. These strategies include lexical, dense, and hybrid retrieval. The retrieved battle representations are then provided to the LLM as external context during the question-answering experiments. For the evaluation, we prepare several query types, including temporal, spatial, participant-based, military, tactical, narrative, outcome-related, and similarity-based queries.

In the final step, we conduct experiments with three different LLM question-answering setups. We compare a baseline LLM that relies only on its internal knowledge with a JSON-augmented LLM that receives structured battle representations as additional context and a raw-text-augmented LLM that uses unstructured battle descriptions from the corresponding Wikipedia articles as context. Our experiments cover both retrieval and answer generation. We first compare the retrieval strategies to determine which one is most effective for selecting relevant battle representations. The selected retrieval strategy is then used in the downstream question-answering experiments. Then, the three experimental setups are evaluated using predefined metrics.

1.5 Contributions

The main contributions of this thesis are the following:

1. We design a domain-specific structured representation schema for historical battles that combines factual and military information with narrative aspects such as ordered events, turning points, participant tactics, action sequences, and narrative patterns.
2. We construct a large-scale structured knowledge base containing 11,561 historical battle representations. The knowledge base is built from Wikidata and English Wikipedia articles and populated automatically using our implemented LLM-based extraction pipeline. We additionally introduce a verification and regeneration stage to improve the consistency and completeness of the generated battle representations.
3. We implement and compare three retrieval strategies for structured battle representations: lexical retrieval with BM25, dense retrieval using embeddings, and hybrid retrieval based on Reciprocal Rank Fusion. We evaluate these methods across multiple query types.
4. We construct an evaluation benchmark of 102 historical battle queries with predefined gold battle sets, query types, context rarity, and closed-set negative examples. This benchmark is used to evaluate both retrieval and answer-generation performance.
5. We evaluate the effect of structured battle representations on LLM-based question answering through both the Oracle-context experiment, where the LLM is provided with a predefined set of candidate battles to choose from, and the end-to-end RAG experiment. We compare a baseline LLM with raw-text-augmented and JSON-augmented configurations and analyze performance across common and rare battles, different query types, and different retrieval depths.

The source code, knowledge base, and evaluation data are available in the GitHub repository.

Our experiments show that structured JSON context is particularly beneficial for rare-battle queries and mainly improves the model's ability to recover relevant battles. At the same time, the results demonstrate that final end-to-end performance remains strongly dependent on retrieval quality.

2 Related work

In this chapter, we review several previous works that are relevant to our thesis. We discuss papers on narrative and event-centered representations, schema-guided information extraction, retrieval methods, and retrieval approaches for battle-specific information. The main goal of this chapter is to provide the conceptual background for our work, covering the main aspects of our implementation, such as representing battles as structured events, extracting information from web articles, retrieving relevant battles, and evaluating the final question-answering pipeline.

2.1 Narrative Representations

In this thesis, we treat historical battles as event structures. They contain both factual information, such as date, location, participants, and outcome, and event-centered narratives that involve the temporal development of actions, each side's objectives, turning points, and consequences. Several previous works support the idea of representing a battle instance as a structured event containing these types of information.

Chambers and Jurafsky (2010) motivates the use of structured event knowledge for document-level understanding tasks such as information extraction, coreference, summarization, and inference. Their work argues that modeling event participants and temporal relations, rather than treating events as isolated facts, can improve narrative understanding.

Another work on Event Knowledge Graphs (EKGs) (Guan et al., 2023) discusses event-centered knowledge representations that capture different aspects of events, such as participants, temporal development, and relations between events.

Hamilton et al. (2026) also supports the use of broader narrative structures for narrative understanding, particularly temporal aspects such as the order and duration of events.

Overall, these ideas motivate the design of our battle representation schema. In our case, we do not construct a knowledge graph; instead, we follow similar event-centered principles and organize the extracted information within a predefined JSON representation schema. Our battle representations therefore include both quantitative characteristics and narrative-rich information, making them suitable for simple factual questions as well as more complex analyses of narratives, tactics, and outcomes.

2.2 Schema-guided event extraction

After we define the representation schema, the next challenge is to extract the required information from raw text and map it into the predefined JSON schema. We use Wikipedia articles as the source of unstructured information about historical battles. The process of extracting historical battle information is related to event extraction (Lai, 2022) and schema-guided knowledge extraction (Luo et al., 2025), where information about events, participants, and relations is extracted from raw documents and organized according to a predefined structure. In this subsection, we describe three relevant works that provide useful ideas for our approach.

We define a historical battle instance as an event, so we plan to extract it in the same way as other event structures. Lai (2022) describes event extraction as an information extraction task that identifies events in text and organizes them into predefined structures or classes. The survey also suggests describing event information through elements such as participants, roles, time, and location.

Recent work on schema-guided LLM extraction (Luo et al., 2025) presents an approach where LLMs are used to extract structured knowledge from raw text. The work suggests that knowledge extraction can benefit from well-defined schema design, validation, and correction. Such schemas can be represented in JSON format.

Another work on schema-aware event extraction argues that event extraction can support downstream tasks such as question answering and information retrieval (Liang et al., 2025). The work also discusses the problem of "schema hallucination", where LLMs may invent non-existent event types during extraction if they are not sufficiently constrained (Liang et al., 2025).

Following these ideas, we plan to use Wikipedia articles on historical battles to extract key information and populate our predefined JSON schema. As described in the schema-guided extraction approach, we define a JSON schema and then use an LLM to populate it with information extracted from raw Wikipedia articles. The use of a fixed JSON schema with predefined fields is also important for guiding the LLM extraction process and preventing the model from generating arbitrary structures.

2.3 Retrieval Methods and RAG

After we constructed a knowledge base with structured battle representations, the next goal is to build a retrieval pipeline that looks for relevant battles depending on a given query. At this stage, retrieval quality becomes a central challenge. In our work, we investigate different retrieval approaches, including lexical, dense, and hybrid methods, and test them over 11,561 battle instances. In this subsection, we discuss several relevant papers that motivate different retrieval methods used in our experiments.

To choose the most effective retrieval approach, we investigated a work that directly compares traditional sparse retrieval methods such as TF-IDF and BM25 with dense retrieval methods. BM25 is a probabilistic ranking function that scores documents based on term frequency saturation, document length normalization, and inverse document frequency (IDF) to estimate their relevance to a given query (Robertson and Zaragoza, 2009). Conversely, dense retrieval performs information retrieval by mapping questions and passages into a

low-dimensional latent semantic space rather than relying on sparse keyword matching methods such as TF-IDF or BM25 (Karpukhin et al., 2020).

The HybridRAG paper (Sarmah et al., 2024) paper discusses some limitations of classical vector-based RAG and structured graph retrieval. It mentions that vector-based RAG may retrieve semantically similar chunks but can struggle to preserve structured context. Meanwhile, structured graph retrieval can capture relations but may still struggle with broader semantic information. The authors introduce an approach that combines both methods into a hybrid RAG system to improve question answering over complex data.

In our work, we also investigate how lexical and dense retrieval methods perform and evaluate a combined approach called hybrid retrieval. Unlike HybridRAG, we do not work with knowledge graphs. Instead, we work with structured battle representations and combine lexical retrieval with dense retrieval within the RAG pipeline. This allows us to benefit from both lexical matching and semantic similarity and potentially improve the quality of battle retrieval.

2.4 Battle-specific Retrieval

As discussed in earlier parts, battle-specific questions require both factual data and information about ordered actions, temporal development, narratives, tactics, and outcomes. Thus, when we consider retrieval in the historical battles domain, we should focus on the natural structure of a battle. In this subsection, we discuss several works related to event-aware retrieval and battle-specific similarity modeling.

Yang et al. (2025) discuss the limitations of conventional RAG approaches for narrative-rich information. They argue that traditional RAG systems usually process text at the level of documents, passages, or sentences and may therefore fail to capture the underlying event structures that describe real-world narratives. The authors therefore propose organizing the retrieved knowledge into an event-centric representation that captures relations between events and their temporal development. This idea is related to our work, where we also work with narrative-rich historical battles and therefore need the LLM to consider not only factual, but also narrative information.

Another recent work on the historical battles domain showed that there are some limitations in describing battles using only quantitative statistics (Coulson et al., 2023). The authors noticed that simple statistics may miss several aspects of battles, such as relationships between actions and collaboration, so higher-level descriptors are needed to capture such aspects. In their approach, the model learns latent representations of actions and units within battles, which can reveal more insights into similarities between tactical behavior and the strategies of participants. This directly supports our idea that we need to describe a battle with more than just simple factual metadata.

The RAGAS paper (Es et al., 2024) introduces several ideas that are relevant to the evaluation of our RAG pipeline. The paper presents a framework for evaluating RAG pipelines across three quality metrics: faithfulness, answer relevance, and context relevance. These metrics show how well the generated answer is supported by the given context, whether it addresses the initial question, and how relevant the retrieved context is. This is useful for our work because it provides more insight into how well the LLM uses the provided context and whether the context itself is relevant and not too noisy.

3 Methodology

This chapter describes the methodology used to construct and evaluate our historical battle question-answering pipeline, named HybridB. First, we explain how the battle corpus is collected and transformed into structured JSON representations, including the LLM-based population, verification, and regeneration stages. Second, we describe the three retrieval methods used to identify relevant battle instances: lexical, dense, and hybrid retrieval. Finally, we present the answer-generation setup used to compare baseline, JSON-augmented, and raw-text-augmented LLM configurations. The project can be accessed through the following GitHub repository: `https://github.com/Sandora1933/HybridB`

3.1 Data Sources and Knowledge Base Construction

First, we need a centralized source of historical battles. Figure 3.1 shows the complete process of gathering the initial data from the source and constructing our local knowledge base of raw-text articles.

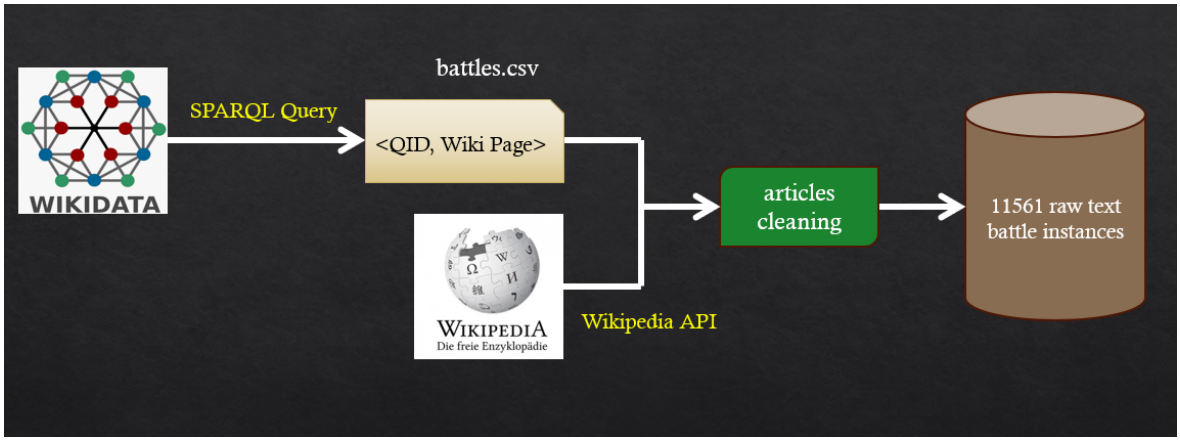


Figure 3.1: Historical battle data collection pipeline.

Wikidata is a central storage system for structured data used across sister projects such as Wikipedia. The Wikidata repository consists mainly of items, where each item has a label, a description, and optionally a number of aliases. Items are uniquely identified by a `Q` followed by a number, for example Battle of Leipzig (`Q151005`). Statements describe characteristics of

an item and consist of a property and a value. Properties in Wikidata are identified by a P followed by a number, such as P31 for *instance of* (Wikidata contributors, 2026). Wikidata provides an efficient way to retrieve historical battle instances using the SPARQL query language. SPARQL is the W3C-standard query language for RDF graph data, which we mainly use to retrieve Wikidata entities and their corresponding historical battle pages on Wikipedia (Harris and Seaborne, 2013). Listing 3.1 shows the SPARQL query used to retrieve all Wikidata battle instances and filter them to include only entities that have a corresponding English Wikipedia page. The predicate wdt:P31 represents the property *instance of*, while the object wd:Q178561 represents the Wikidata item *battle*. In natural language, the query means: *Retrieve all distinct Wikidata entities that are classified as instances of a battle* (Q178561) *and have a corresponding article in the English-language Wikipedia, returning both the Wikidata entity and its Wikipedia article URL.*

```
SELECT DISTINCT ?item ?wikipediaArticle WHERE {
    ?item wdt:P31 wd:Q178561.

    ?wikipediaArticle schema:about ?item;
        schema:isPartOf <https://en.wikipedia.org/>.
}
```

Listing 3.1: SPARQL query for retrieving historical battle entities from Wikidata and their corresponding English Wikipedia articles.

After executing the SPARQL query, the results were processed to create a clean list of battle instances. For each retrieved entity, the QID was extracted from the entity URI and paired with the corresponding English Wikipedia article URL. Duplicate entities were removed using the QID as the unique identifier, and the remaining records were sorted by QID. The resulting dataset was then stored as a CSV file containing one row for each battle instance together with the corresponding English Wikipedia page, as illustrated in Table 3.1.

Table 3.1: Example entries from the collected battle dataset

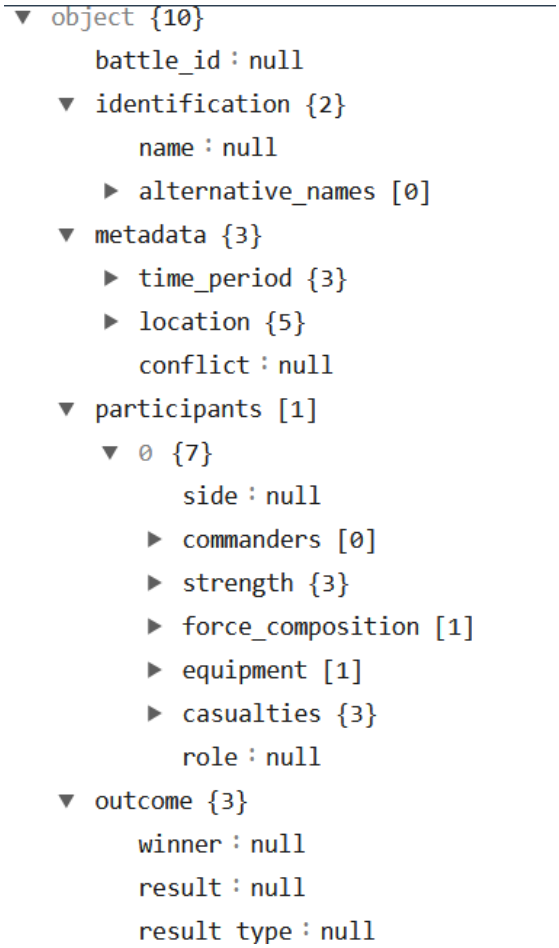
#	qid	wikipedia_url
1	Q330	https://en.wikipedia.org/wiki/Battle_of_Marignano
2	Q1392	https://en.wikipedia.org/wiki/Battle_of_Nieuwpoort
⋮	⋮	⋮
11561	Q140628890	https://en.wikipedia.org/wiki/Battle_of_Bizye_(1256)

Following this step, for each battle instance from the resulting list, we programmatically retrieve the associated Wikipedia article in raw text format using the wikipediaapi Python library (Majlis). The extracted article text contains the page summary and article sections. Non-content sections such as references, external links, citations, and bibliographies are not included. After cleaning, the text of each article is stored locally as a UTF-8 text file named after its Wikidata QID. At this stage, we obtain the raw textual knowledge base containing 11,561 battle instances collected using Wikidata and Wikipedia.

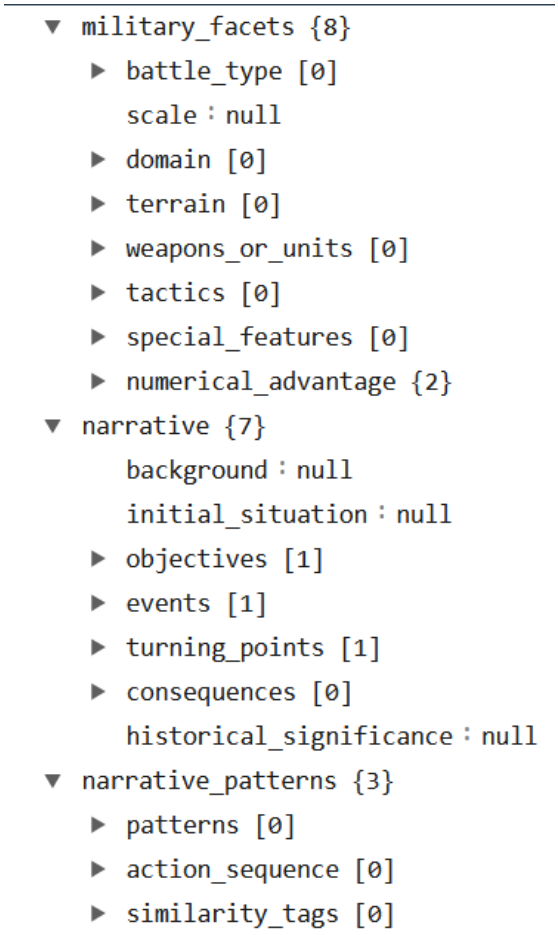
3.2 Battle Representation Schema

Following the ideas discussed in Chapter 2.1, we decided to design a structured battle representation schema that can later be populated with real information from our battle

sources. In general, our representation schema is a JSON structure containing 10 top-level categories and 85 individual fields (leaf fields), covering basic factual information about the battle as well as higher-level aspects such as military facets, narratives, narrative patterns, outcomes, and summaries. Figure 3.2 shows the overall structure of our representation schema.



(a) Identification, metadata, participants, and outcome.



(b) Military facets, narrative, and narrative patterns.

Figure 3.2: Overview of the structured representation schema for historical battles, illustrating its main information categories and nested fields.

When we design a battle representation, we consider relevant aspects that can later help the LLM with reasoning. We start with `battle_id` and identification fields to be able to distinguish between battles during debugging. Apart from battle names, we also include alternative names for cases where a battle is known under different names. For instance, in Wikidata, the Battle of Leipzig (`Q151005`) is also known as the Battle of the Nations.

Next, we define metadata fields such as `time_period`, `location`, `conflict`, and `country_modern`. The `time_period` includes a start date and an end date in ISO format, for example `1515-09-13`, and additionally a human-readable date label such as *13 September 1515*. `country_modern` is a field that describes the modern country located on the territory where the battle occurred. It can be useful for queries such as *Describe battles involving a double envelopment that occurred in the 16th century on the territory of modern Germany.* To be precise, `country_modern` contains several edge cases where the modern country is difficult to define, for example when a battle occurred across the territory of several modern countries, as in the Battle of

the Bulge (1944–1945), or when the modern country is not universally recognized, as in the case of Kosovo for the Battle of Kosovo (1389).

The next aspect is participants. Here, information about sides, commanders, strength, force composition, and other related details is captured. Each side includes a list of commanders who participated in the battle, while the `strength` field contains the number of troops, including different troop types. For instance, the total strength can be 3,100 with the label *1,100 knights, 2,000 infantry*. If a side is a coalition of several nations, we explicitly define the strength of each participant in the note. For example, for the Coalition side in the Battle of Leipzig: *Estimates of 380,000 troops with 1,500 guns; breakdown: Russians 145,000–150,000, Austrians 115,000, Prussians 75,000–90,000, Swedes 19,000–30,000.* Then, for each contingent involved on each side, we describe the force composition in detail, specifying its size, unit types, equipment, casualties, and role in the battle. This information is relevant for queries such as *Which battles involved a coalition of multiple nations fighting against a numerically superior enemy?*, where the model needs information about the coalition sides and the sizes of their forces. Besides, equipment and unit types can be very useful when we want to ask the model which side had more tanks or artillery but did not gain a terrain advantage and still lost the battle.

To be able to reason over battle outcomes and obtain more concrete insights into the results, we include an additional `outcome` field that specifies the `winner`, `result`, and `result_type`. The `result` field describes the nature of the outcome. It can represent, for example, a *decisive victory*, a *successful defense*, a *successful withdrawal*, or an inconclusive result when it is difficult to summarize the outcome clearly. The `result` is expressed from the perspective of the side identified as the `winner`, when applicable. The `result_type` field describes the significance of the outcome. For instance, *tactical* means an immediate battlefield success, *operational* refers to a success affecting a campaign or series of battles, while *strategic* describes a broader success affecting overall war objectives. Naturally, such classifications can be subjective and depend on interpretation. These questions, together with possible future improvements, are discussed in Chapter 6, Discussion.

Next, we describe the military facets, which capture the key military characteristics of a battle. They include fields such as `battle_type`, `scale`, `domain`, `terrain`, `weapons_or_units`, `tactics`, and `special_features`. Most of these fields are represented as lists and can therefore contain multiple descriptions, while scale is represented as a single value. The `battle_type` field describes the general type of engagement, for example a land battle. The `scale` field indicates the overall size of the battle. The `domain` field specifies whether the battle took place on land, at sea, in the air, or across multiple domains. The `terrain` field describes the main environmental conditions that may affect the battle, such as urban, riverine, mountainous, or village terrain. `weapons_or_units` captures important types of troops and military equipment involved in the engagement, for example infantry, cavalry, artillery, or rockets. The `tactics` field describes important tactical approaches used during the battle, such as encirclement, defense of bridges, or urban street fighting. Finally, `special_features` is used to store particularly distinctive characteristics of a battle, for example the use of Congreve rockets or a Grand Battery of 150 guns in the case of the Battle of Leipzig. These fields are important for queries that require reasoning over the military characteristics of battles, for example comparing battles with similar terrain, identifying battles where particular weapons or tactical approaches were used, or finding battles with unusual military features. One example of such a query is the following: *Which land battles involved fighting in urban areas and near rivers, where commanders employed encirclement tactics and concentrated artillery batteries?*

To improve narrative reasoning, we define a narrative aspect of a battle. It shows how the

battle developed over time and describes individual events in chronological order, as well as how they are connected to the broader course of the engagement. The `narrative` top-level field includes `background`, `initial_situation`, `objectives` of each side, `events`, `turning points`, `consequences`, and `historical_significance`. Each event includes its order number, phase, event type, actors, targets, description, outcome, and optionally its causal role and confidence. This makes our representation able to capture important insights about the order of events and involved actors, and to assign a specific role, target, and outcome to each event. This may help to understand whether a specific side had an advantage over the whole duration of the battle or only at a specific moment. We can also gain more insight into how the engagement developed over time and how the winner ultimately achieved victory. The `turning_points` field helps to define important moments that significantly changed the course of the battle, for example the Coalition encirclement of the French army during the Battle of Leipzig. We also define a `consequences` field that captures the main political and military outcomes after the battle. `historical_significance` additionally provides information about the broader importance of the battle. Good examples of queries that may require this narrative aspect are queries asking to find battles involving an initially successful defense followed by encirclement and retreat, or battles where a tactical turning point later caused a broader strategic defeat.

Apart from the narrative aspect, we additionally define a `narrative_patterns` field that captures higher-level narrative structures. It contains battle `patterns`, an `action_sequence`, and `similarity_tags`. The `patterns` field describes tactical and narrative characteristics of the battle, for example encirclement, urban street fighting, or the use of massed artillery. The `action_sequence` field describes the main stages of the battle in a simplified chronological form. We additionally define a `similarity_tags` field to explicitly mark comparable characteristics that might be useful when trying to find similar battles, for instance large-scale coalition battles, defeats of Napoleon, or battles involving multi-nation forces.

In the `retrieval_texts` field, compact textual summaries are described. They are generated from the structured battle representation and are designed to support retrieval. Specifically, it contains a `tactical_summary`, `narrative_summary`, and `action_sequence_summary` that partially duplicate values defined in the narrative field. The initial motivation for introducing the `retrieval_texts` field was to provide condensed, semantically informative descriptions of battles that could improve dense retrieval without requiring embedding models to process the entire JSON representation. However, as described later in the Retrieval chapter, our implemented retrieval approach serializes and embeds complete JSON documents without separately extracting or prioritizing these summaries. Therefore, although the generated summaries contribute to the input representation, their independent impact on retrieval and answer generation quality remains unknown.

Finally, we added `extraction_metadata` to our representation schema. It stores information about the extraction process and possible uncertainty in the generated battle representation. It contains fields such as `wikidata_id`, `overall_confidence`, and `uncertainty_notes`. The `field_confidence` field is intended to indicate how reliable the extracted information is for most top-level aspects. However, in the current implementation, the filler LLM is instructed to set both `overall_confidence` and `field_confidence` values to `null`. Uncertain or conflicting information from the Wikipedia source is represented through labels, ranges, and the `uncertainty_notes` field. This is especially useful for avoiding the need to force a single value, for instance for troop strength or the number of casualties. The field can contain information such as *French total strength estimates range from 198,000 to 225,000.*

The schema fields, their descriptions, and data types are also provided in Table 3.2.

Table 3.2: Overview of the structured battle representation schema

Battle aspect	Schema fields	Description	Datatype
Battle ID	`battle_id`	Unique identifier of the battle instance, typically corresponding to its Wikidata QID	String / null
Identification	`identification.name`, `identification.alternative_names`	Stores the primary battle name and possible alternative names.	Object
Metadata	`time_period`, `location`, `conflict`; `start_date`, `end_date`, `country_modern`	Contains basic factual metadata about when and where the battle occurred and the wider conflict it belongs to.	Object
Participants	`side`, `commanders`, `strength`, `force_composition`, `equipment`, `casualties`, `role`	Represents the participating sides and their commanders, force size, composition, equipment, casualties, and role in the battle.	`Array⟨Object⟩`
Outcome	`winner`, `result`, `result_type`	Describes the final result of the battle, including the winning side and the type of victory or outcome.	Object
Military facets	`battle_type`, `scale`, `domain`, `terrain`, `weapons_or_units`, `tactics`, `special_features`, `numerical_advantage`	Captures important military characteristics of the battle such as terrain, tactics, force types, weapons, and numerical advantage.	Object
Narrative	`background`, `initial_situation`, `objectives`, `events`, `turning_points`, `consequences`, `historical_significance`	Represents how the battle developed, including its background, objectives, sequence of events, key turning points, consequences, and historical importance.	Object
Narrative patterns	`patterns`, `action_sequence`, `similarity_tags`	Stores higher-level recurring patterns and abstract action sequences that can support comparison between battles.	Object
Retrieval texts	`short_summary`, `tactical_summary`, `narrative_summary`, `action_sequence_summary`, `similarity_query_text`	Contains compact textual representations of the battle designed to support different retrieval and similarity tasks.	Object
Extraction metadata	`wikidata_id`, `overall_confidence`, `field_confidence`, `uncertainty_notes`	Stores provenance and confidence information about the extracted structured representation and notes possible uncertainty.	Object

3.3 Wikidata and JSON Schema Correspondence and Coverage

In this section, we analyze the structural and semantic correspondence between Wikidata properties and the fields of our JSON representation schema. Figure 3.3 illustrates the Wikidata property corresponding to each battle aspect together with its coverage. Wikidata property coverage is defined as the percentage of Wikidata battle instances that have property P. Additionally, we describe the corresponding JSON field and its coverage. JSON field coverage for field F is defined as the percentage of battle records in the local structured knowledge base having a non-empty value for field F. In the final column, we assign a comparison class: *Direct* for closely semantically equivalent properties and fields, *Partial* for related but not identical representations, and *Not applicable* when either Wikidata or our schema does not represent the corresponding battle aspect. The Wikidata battle set contains 18,673 instances, while the local structured knowledge base contains 11,561 battle records. The complete comparison table is available in the root directory of the project's GitHub repository as `wikidata_json_comparison.docx`.

Information Aspect	Wikidata property	Wikidata Property Coverage	Corresponding JSON field	JSON Field Coverage	Comparison class
Battle classification	P31 (instance of)	100%	military_facets.battle_type	80.13%	Partial
Country	P17 (country)	63.44%	metadata.location.country; metadata.location.country_modern	85.02%	Partial
Participants / belligerents	P710 (participant)	32.30%	participants[].side; participants[].force_composition[].contingent	99.62%	Partial
Larger conflict / campaign	P361 (part of)	65.17%	metadata.conflict	73.63%	Partial
Location	P276 (location)	64.17%	metadata.location.name	98.07%	Direct
Date	P585 (point in time)	61.66%	metadata.time_period	100%	Partial
Start date	P580 (start time)	20.00%	metadata.time_period.start_date	99.84%	Direct
End date	P582 (end time)	19.43%	metadata.time_period.end_date	99.68%	Direct
Duration	P2047 (duration)	0.11%	metadata.time_period.duration_days	71.58%	Direct
Time period	P2348 (time period)	1.77%	metadata.time_period	100%	Partial
Coordinates	P625 (coordinate location)	47.37%	No equivalent		Not applicable
Administrative region	P131 (located in the administrative territorial entity)	2.94%	metadata.location.region	77.88%	Partial
Commanders	P4791 (commanded by)	0.05%	participants[].commanders	91.19%	Direct
Participant strength	P1132 (number of participants)	0.12%	participants[].strength.total	39.43%	Partial
Force composition	P4220 (order of battle)	1.57%	participants[].force_composition	74.94%	Partial
Casualties	P1590 (number of casualties)	0.04%	participants[].casualties.total	34.33%	Partial
Deaths	P1120 (number of deaths)	0.60%	participants[].casualties.note	60.66%	Partial
Injured / wounded	P1339 (number of injured)	0.15%	participants[].casualties.note	60.66%	Partial
Significant person	P3342 (significant person)	0.10%	participants[].commanders	91.19%	Partial
Battle / operation target	P533 (target)	0.10%	narrative.events[].targets	87.27%	Partial
Objectives / goals	P3712 (has goal)	0.01%	narrative.objectives[].objective	88.78%	Partial
Causes	P828 (has cause)	0.14%	narrative.background; narrative.events[].causal_role	98.63%	Partial
Effects / consequences	P1542 (has effect)	0.25%	narrative.consequences	95.0%	Partial
Constituent parts	P527 (has part(s))	0.43%	narrative.events	97.91%	Partial
Previous related event	P155 (follows)	0.62%	No equivalent between battle records		Not applicable

Figure 3.3: Comparison of Wikidata properties and corresponding JSON fields.

3.4 Factual Validation of the JSON Knowledge Base

In this section, we additionally assess the source-grounded factual correctness of our generated JSON knowledge base with respect to the corresponding Wikipedia sources. For this

purpose, we perform a small-scale validation experiment. We select 20 well-represented battles and sample four facts from each JSON battle representation, resulting in 80 facts in total. We include four battle aspects: metadata, participants, outcome, and military facets, and select one fact from each category using a fixed random seed. Table 3.3 shows the experiment configuration.

Table 3.3: Configuration of the JSON knowledge base validation experiment

Parameter	Configuration	
Reference source	English Wikipedia raw article text	
Number of battles	20	
Facts per battle	4 (one per category)	
Generated fact count	80	
Random seed	42	
Fact categories	`metadata`, `participants`, `outcome`, `military_facets`	
Selected battles	**Battle name and QID**	**Battle name and QID**
1–2	Battle of Leipzig (Q151005)	Battle of Stalingrad (Q38789)
3–4	Battle of Waterloo (Q48314)	Battle of Zama (Q200056)
5–6	Battle of Wuhan (Q2038907)	Battle of Gettysburg (Q33132)
7–8	Battle of Borodino (Q184320)	Battle of Vienna (Q200855)
9–10	Battle of Hastings (Q83224)	Battle of Cannae (Q179591)
11–12	Battle of Marathon (Q31900)	Battle of Agincourt (Q188495)
13–14	Battle of Austerlitz (Q134114)	Battle of the Somme (Q132568)
15–16	Battle of Iwo Jima (Q180182)	Battle of Thermopylae (Q131969)
17–18	Battle of Okinawa (Q192660)	Battle of Berlin (Q154182)
19–20	Battle of Tannenberg (Q153858)	Battle of Tours (Q173077)

After that, each sampled fact is validated by an LLM against the source text of the corresponding Wikipedia article. The validation model is instructed to use only the source article and classify each fact as *supported*, *contradicted*, or *not found*. *Supported* means that the article provides sufficient evidence for the claim, *contradicted* means that the source contains information incompatible with the claim, and *not found* means that the source does not provide sufficient evidence either to support or contradict the claim. The generated facts and validation results are stored in the `results/fact_validation` directory. The results of this validation experiment are discussed in Section 5.4 of the Experimental Results chapter.

3.5 LLM-Based Schema Population

At this stage, we already have a raw-text knowledge base containing 11,561 historical battle articles and a JSON representation schema covering the relevant aspects of a battle. The next step is to use an LLM to populate the JSON schema for each raw-text battle article. As a result, we obtain a structured knowledge base consisting of 11,561 JSON files, one for each battle instance.

First, we introduce the GPT-OSS-120B model, which was used for the schema population task and also for almost all other LLM-based tasks described later (OpenAI, 2025). The model was accessed through the ScaDS.AI service via its OpenAI-compatible REST API (ScaDS.AI Dresden/Leipzig). Communication with the service was implemented using the official OpenAI Python client and configured with the ScaDS.AI endpoint. Deterministic decoding was used by setting the temperature to 0.0 and disabling sampling. The maximum number of newly

generated output tokens was set to 4,000. Lower values sometimes caused incomplete generations and parsing errors, which motivated the use of a 4,000-token limit. The model response was returned as plain text and subsequently processed by the experiment pipeline. We treated responses terminated due to the maximum token limit as generation failures. Likewise, responses with missing content were handled as invalid model outputs.

Next, given the LLM gateway and the local knowledge base of battle articles, we proceed with designing a prompt for the schema-population task. The idea is to ask the model to fill the empty representation schema with values based on the raw-text article. Figure 3.4 shows the prompt structure without the 17 additional rules, while all 17 rules included in the prompt are presented below.

```
You are extracting structured information about one historical battle.

TARGET BATTLE QID:
{qid}

TASK:
Complete the JSON template using the supplied source text.

Extract as much source-supported information as possible for all relevant
schema fields, including formations, commanders, tactics, equipment details,
numerical estimates, subunit casualty figures, battle phases, turning points,
and uncertainty notes.

Do not invent information, use external knowledge, or make unsupported
assumptions.

RULES: [...]
Before returning, silently verify that:
- every populated value is supported by the source;
- no fact from another battle was assigned to the target battle;
- participant totals do not represent only partial forces;
- subunit casualties are not presented as whole-side casualties;
- uncertainty and conflicting estimates are preserved;
- all schema types and array structures are unchanged;
- no empty placeholder objects remain;
- the output is valid JSON.

EMPTY JSON TEMPLATE:

{schema_text}

SOURCE TEXT:

{battle_text}

Return the completed JSON object.
""".strip()
```

Figure 3.4: Prompt template used to guide GPT-OSS-120B in extracting source information from Wikipedia articles and populating the structured JSON battle representation schema.

Extraction Rules

1. Use only information supported by the source text. Do not use external or prior knowledge.
2. Extract information only about the target battle and its directly relevant background, course, outcome, and consequences. Do not assign facts from other battles, unrelated campaign events, or later historical developments to the target battle unless the source explicitly connects them to it.
3. Preserve the exact JSON structure, field names, nesting, data types, and array item types. Do not add, remove, rename, or restructure fields.
4. Fill as many fields as the source supports.
   - Unsupported scalar fields must be `null`.
   - Unsupported arrays must be `[]`.

- Do not return placeholder array objects containing only null or empty values.
- Do not fill a field merely to make the JSON appear complete.

5. Set `battle_id` and `extraction_metadata.wikidata_id` to the provided QID.
6. Preserve uncertainty, ranges, approximations, alternative estimates, and conflicting figures. Do not replace them with unsupported exact values. Use `label`, `note`, and `uncertainty_notes` where appropriate.
7. Do not calculate totals unless all components are explicit, complete, unambiguous, and refer to the same side in the target battle. Explain any calculation in the corresponding `note` field.
8. A participant strength total must represent the complete supported strength of that side. Do not use the number of one component, such as infantry, as the side's total strength when additional forces are also mentioned.
9. Include commanders only when the source clearly connects them to command during the target battle. Preserve major allies and supporting contingents that actually participated.
10. Keep target-battle casualties separate from casualties belonging to other battles, the wider campaign, pursuit, retreat, or later aftermath.
11. Preserve supported subunit casualty figures, but clearly associate them with the relevant formation or event. Do not present subunit casualties as the total casualties of an entire participant side.
12. Distinguish equipment used in the battle from captured materiel. Do not treat captured equipment as pre-battle strength.
13. Concise source-grounded abstraction is allowed for military facets, narrative events, turning points, narrative patterns, similarity tags, and retrieval summaries. Do not invent unsupported facts, motives, causal claims, or specialized terminology.
14. When the source describes a clear battle sequence, populate chronological narrative events, important turning points, action sequence, and retrieval summaries. Focus on developments important to the overall battle while preserving significant supported details.
15. Dates:
    - Use `YYYY-MM-DD` for CE `iso_date` when day, month, and year are known.
    - Populate `duration_days` only when both boundary dates are precise.
    - Count duration inclusively.
    - Do not create non-standard ISO strings for BCE dates.
    - Record important date inconsistencies in `uncertainty_notes`.
16. Set all confidence fields to `null`.
17. Return exactly one valid JSON object. Do not include Markdown, comments, explanations, or text outside the JSON.

In the prompt, the LLM was given a role, a specific task, a set of rules, an output format, and the attached raw-text article together with the empty representation schema.

The 17 rules provided in the prompt are designed to constrain the schema-population process and reduce unsupported outputs. The rules can be divided into several groups based on source grounding, structural consistency, uncertainty handling, and the special interpretation of battle-specific numerical information.

One of the most important rules (Rule 1) forces the model to use only information supported by the source text, without relying on its internal knowledge. This makes the result traceable to the corresponding Wikipedia article. A similar rule (Rule 2) restricts extraction to the target battle, preventing facts from similar battles or the wider campaign from being incorrectly assigned to the target instance.

Among the structural consistency rules, it is worth mentioning Rule 3, which requires the

model to preserve the initial JSON structure provided in the `empty_schema.json` file. This guarantees that all generated JSON files follow the same pattern. Additionally, Rule 4 instructs the model to fill fields only when they are directly supported by the source and to use `null` or empty arrays for unsupported information instead of inventing details.

Another group of rules addresses uncertainty and numerical data. Rule 6 instructs the model to preserve value ranges, approximations, alternative estimates, and conflicting figures when no exact value is provided. Rule 7 also states that the model should not calculate totals unless all components are explicit, complete, and unambiguous, which is especially important for troop strength and casualty numbers.

Several rules focus on the narrative representation. In particular, Rules 13 and 14 instruct the model to populate chronological events, turning points, action sequences, and retrieval summaries when they are supported by the source, while still avoiding unsupported claims, causal statements, or invented tactical terminology.

Finally, Rule 17 requires the model to return exactly one valid JSON object without Markdown, comments, or additional explanations.

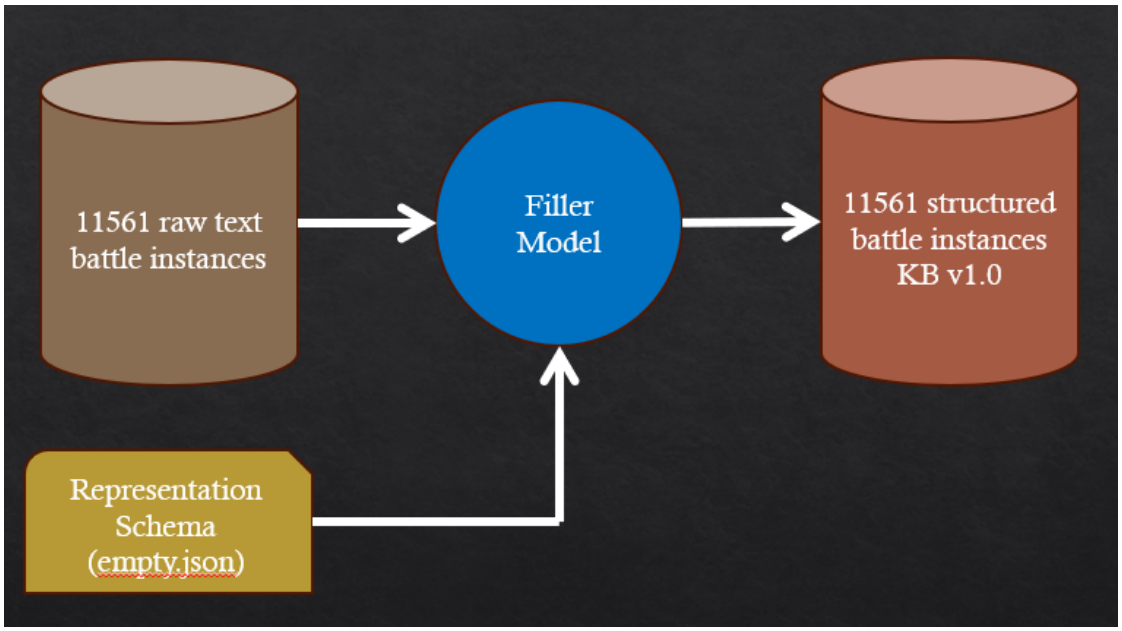


Figure 3.5: Construction of the structured historical battle knowledge base (KB v1.0).

As a result, after running the generation pipeline over the entire raw-text knowledge base, we obtain a knowledge base of JSON battle representations, as illustrated in Figure 3.5. The resulting KB is available in the project folder under `data/json_kb.zip`. It is worth mentioning that this is only the first version of our JSON knowledge base. In the following section we introduce an additional pipeline with an LLM verifier and a regeneration step, which finally leads to the second version of the JSON knowledge base that is expected to have higher quality.

3.6 LLM Verification and Regeneration

During the analysis of the generated JSON battle representations, we identified several cases where the first version of the knowledge base (KB1) contained incomplete or semantically

inconsistent structured information. One example is the Battle of Stalingrad. In KB1, the Soviet force composition contained only a subset of the formations explicitly mentioned in the source article, including the 62nd, 51st, 57th, 24th, and 5th Tank Armies. However, the source also described several additional Soviet formations involved in the later encirclement operation, such as the 28th, 64th, 65th, 66th, 1st Guards, and 21st Armies, as well as several air armies. After verification and regeneration, these missing formations were added in KB2, resulting in a more complete representation of the Soviet forces. The comparison between the corresponding KB1 and KB2 fields is shown in Figure 3.6.



Figure 3.6: Comparison of the Soviet force composition fields in KB1 and KB2.

A second example concerns the Battle of Waterloo. In KB1, the role of the Prussian participant was classified as defender. This is inconsistent with the source article, which describes the arriving Prussian forces actively attacking the French flank and advancing toward Plancenoit during the battle. After verification and regeneration, the participant role was corrected to attacker in KB2. Unlike the Stalingrad example, which mainly demonstrates an improvement in completeness, the Waterloo case shows how the verification step can correct the semantic interpretation of a structured field. Figure 3.7 illustrates this change between KB1 and KB2.

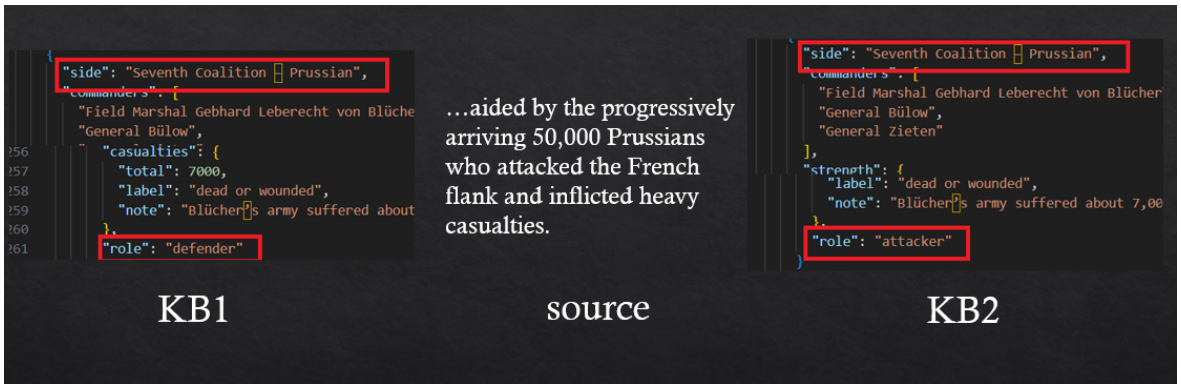


Figure 3.7: Comparison of participant role of Prussian forces in KB1 and KB2.

The design of the verification and regeneration pipeline that led to KB2 is shown in Figure 3.8. First, for each battle, the generated KB1 JSON record was paired with its corresponding raw Wikipedia article, which served as the reference source. In the first stage, an LLM verifier compared the structured representation against the source text and identified unsupported

information, factual inconsistencies, schema-semantic errors, and important omissions. The verifier returned a structured report containing either a `PASS` verdict or a `REVISE` verdict with a list of recommended corrections.

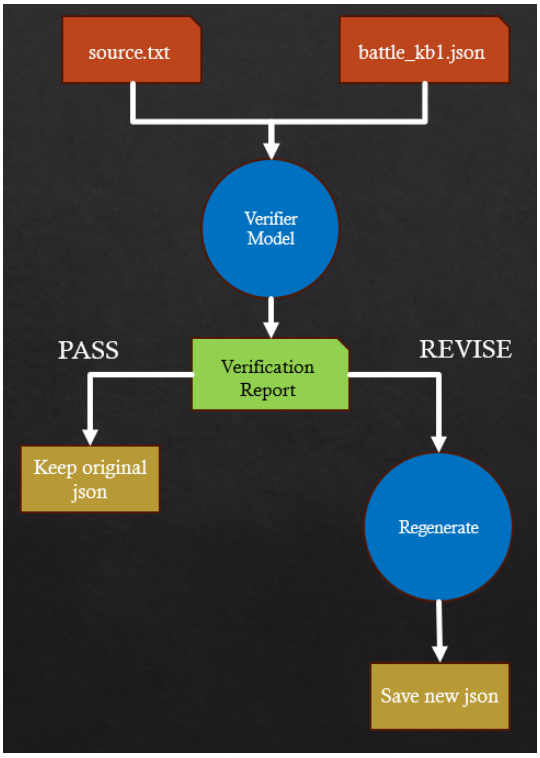


Figure 3.8: Knowledge base verification and refinement pipeline.

Records classified as `PASS` were retained unchanged. For records classified as `REVISE`, a second LLM call was performed. As the verification model, we again used GPT-OSS-120B. The regeneration model, which was again GPT-OSS-120B, received the original JSON record, the raw source text, and the verifier report. It was instructed to apply only source-supported corrections while preserving valid information and the original schema structure. The raw article remained the single source of truth. The regenerated output was then parsed and validated before being stored in the second version of the knowledge base, KB2. To improve robustness, both verification and regeneration calls were retried up to three times in case of parsing or generation failures. Invalid verifier reports and structurally inconsistent regenerated JSON objects were rejected.

As a result, we obtained the second version of the knowledge base, with improved consistency and factual correctness. During our experiments, we compare both versions to determine whether the second version of the knowledge base improves the performance of the model and the RAG system. For most experiments, KB2 is used.

3.7 Retrieval Methods

In our future experiments, we are going to compare the performance of a baseline LLM with an LLM augmented with JSON context. Therefore, our next goal is to build a retrieval pipeline that retrieves the top-k most relevant battles from our local knowledge base containing 11,561 JSON battle instances. According to RQ2, we want to compare the performance of three different retrieval methods: lexical, dense, and hybrid retrieval, where hybrid retrieval

combines the two methods mentioned above. For our end-to-end generation experiments, we are going to choose the retrieval method that performs best. In this chapter, we discuss all three retrieval methods, how they are implemented in our work, and how each of them is useful in our case. The overall implementation of the retrieval pipelines is illustrated in Figure 3.9.

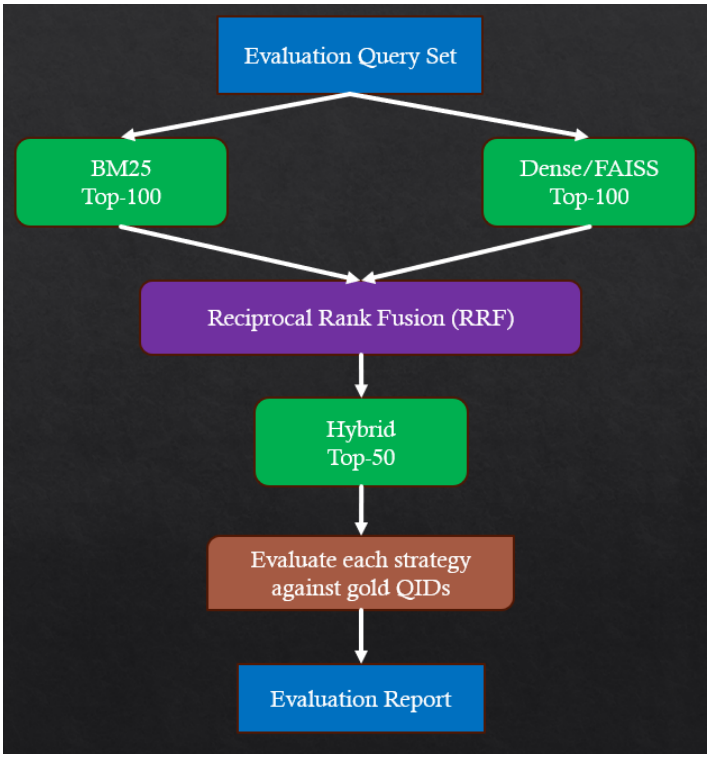


Figure 3.9: Hybrid retrieval and evaluation pipeline.

3.7.1 Lexical Retrieval with BM25

First, we decided to include a lexical retrieval algorithm as part of our retrieval pipeline. Questions about historical battles often require information about exact names, locations, commanders, and tactical terms that may overlap with the corresponding battle representation. In such cases, a lexical retrieval algorithm such as BM25 can efficiently identify relevant battle documents based on term matching. It is also relatively simple and interpretable, which makes it useful as a baseline for testing.

BM25 is a widely used lexical retrieval algorithm that ranks documents according to their relevance to a given query based on term frequency, inverse document frequency, and document length normalization (Robertson and Zaragoza, 2009). Term frequency reflects how often a query term appears in a document. Inverse document frequency gives more weight to terms that appear in fewer documents. Document length normalization is another factor in a ranking function that adjusts term frequency based on how long a document is compared to the average document length. The relevance score of a document d for a query q is calculated using the BM25 scoring function, as shown in Equation 3.1.

$$\mathrm{BM25}(q,d) = \sum_{t \in q} \mathrm{IDF}(t) \cdot \frac{f(t,d)(k_1+1)}{f(t,d) + k_1 \left(1 - b + b\frac{|d|}{\mathrm{avgdl}}\right)} \tag{3.1}$$

The function $\mathrm{IDF}(t)$ measures the importance of a term based on its rarity across the document collection, while $f(t, d)$ represents its frequency in document d. The parameter k_1 controls term frequency saturation, and b determines the degree of document length normalization. Finally, $|d|$ denotes the document length, and avgdl represents the average document length across the collection.

In this work, lexical retrieval was implemented using the `BM25Okapi` algorithm provided by the Python `rank_bm25` library (Brown, 2021). The implementation follows the classical BM25 ranking described by Robertson and Zaragoza (2009).

We treat each battle instance as a single retrieval document. Each structured JSON representation is serialized into a raw-text form and then tokenized by lowercasing the text and extracting tokens. The same preprocessing is applied to the user query. After the resulting corpus is indexed by the BM25 algorithm, the retrieval function computes a relevance score for every battle instance. The documents are then sorted in decreasing order of relevance, and the top-k candidates are returned. Our implementation uses the default parameters provided by the library.

We index the complete JSON document rather than focusing on specific fields in order to search for lexical matches across all fields, including participants, outcomes, tactics, and others. In our implementation, BM25 does not distinguish between different JSON fields and therefore treats the entire document, including both field names and field values, as a flat token sequence. We later discuss this as a limitation and explore possible ways to make BM25 more structure-aware. This is discussed in Chapter 6, Discussion.

A successful lexical retrieval example was obtained for the query *Find major battles fought in 1066 during the struggle for the English throne.* As a result, the first rank assigned by the BM25 algorithm was the Battle of Hastings (`Q83224`), with a relevance score of 44.52. Table 3.4 shows an inspection of the relevance signals. It illustrates that the strongest signal was given by the year *1066*, followed by *English* and *throne*. These terms were matched across different components of the structured battle representation, including metadata, participant information, and narrative content. This example demonstrates how BM25 can be effectively used for battle representations when there is substantial lexical overlap between a query and a relevant document, even without explicitly understanding the underlying nested JSON structure.

Table 3.4: Contribution of query terms to the BM25 retrieval score

Query term	Contribution	Why it matters
`1066`	14.71	Exact temporal match
`english`	7.34	Matches the historical and geographical context
`throne`	4.70	Matches the succession conflict described in the narrative
`major`	0.67	Matches the battle's scale classification

Although BM25 performs accurate retrieval in several cases, it still has some limitations. In particular, the algorithm relies on exact term matching and may struggle with questions that contain relevant concepts but use different wording from the corresponding passages. Furthermore, our lexical retrieval implementation does not explicitly exploit the underlying JSON structure and operates on the entire serialized document. Therefore, it may match information from less relevant fields and assign weight to tokens that are not useful for a particular query. We discuss this limitation in more detail in Chapter 6.

To address the limitation of relying on exact wording and to capture relevant concepts even when there is no direct lexical overlap, we also investigate a dense retrieval approach.

3.7.2 Dense Retrieval

As discussed in the Related Work, dense retrieval performs information retrieval by mapping queries and passages into a low-dimensional latent semantic space rather than relying on sparse keyword-matching methods such as BM25. Dense retrieval maps queries and passages into a shared embedding space and allows matches between semantically similar expressions even when they do not share the same tokens. For example, a query that mentions a *surrounded army* may still match a passage that contains the term *encirclement*. Similarly, a query asking about a *withdrawal under pressure* may match passages containing the term *retreat*. This is especially useful for battle representations that share the same tactical or narrative idea but express it in different ways. This makes dense retrieval better suited for capturing higher-level semantics and similarities between battles, and particularly useful for queries such as *Find battles similar to the Battle of Cannae in which encirclement or converging attacks trapped a large enemy force and led to a catastrophic defeat*.

Karpukhin et al. (2020) formulates dense retrieval as a similarity comparison between independently encoded query and document representations. First, a query q and a passage p are mapped to dense vectors by separate encoders E_Q and E_P.. Their similarity is then calculated using the inner product:

$$\text{sim}(q, p) = E_Q(q)^T E_P(p) \tag{3.2}$$

Documents with higher similarity scores are considered more relevant to the initial query. The original formulation provides a theoretical basis for our implementation, although in our approach both query and battle representation embeddings are L2-normalized before retrieval. As a result, the inner product becomes equivalent to cosine similarity:

$$\text{sim}(q, d) = \frac{e_q \cdot e_d}{\|e_q\| \|e_d\|} \tag{3.3}$$

In our work, we used the `Qwen/Qwen3-Embedding-4B` embedding model provided through the ScaDS.AI OpenAI-compatible endpoint to build an index of dense vectors for our battle representations. Each battle record is represented by serializing the complete JSON object into compact text and computing a dense embedding. Embeddings were generated in batches of eight and stored as 32-bit floating-point vectors in the vector index.

The normalized battle embeddings are stored in a FAISS IndexFlatIP index (Douze et al., 2024), which performs exact nearest-neighbor search. When a query is executed, it is first converted into a normalized embedding vector. The query vector is then compared with all indexed battle embeddings using inner-product similarity. Finally, the top-k battle records with the highest similarity scores are returned.

However, dense retrieval also has several limitations. It may struggle to retrieve relevant battle representations based on exact entity names, rare military terms, and other specific lexical signals that are not reliably represented in the semantic embedding space. In contrast, lexical retrieval is especially effective in cases where exact or overlapping keywords are important.

During our retrieval experiments, we observed an interesting case when testing our lexical and dense retrieval pipelines on a query asking for battles in which war elephants participated in a major attacking charge that was stopped by infantry, artillery, or reserve forces and failed to secure victory. As a result, the Battle of Khajuha (`Q4871417`), which belongs to the gold set, was ranked 3rd by the BM25 method, whereas dense retrieval placed it only in 75th position. During the analysis of term frequency and other factors used by the BM25 algorithm, we found that several terms, such as *elephants*, *reserve*, *war*, and *infantry*, made the largest contribution to the final score, which helped rank the Battle of Khajuha (`Q4871417`) so highly, as illustrated in Figure 3.10. Table 3.5 additionally shows how the overall contribution of each term was calculated based on the different scoring factors. These terms directly correspond to structured information contained in the battle representation and can be found in several battle aspects, including force composition, action sequence, narrative patterns, and narrative summary.

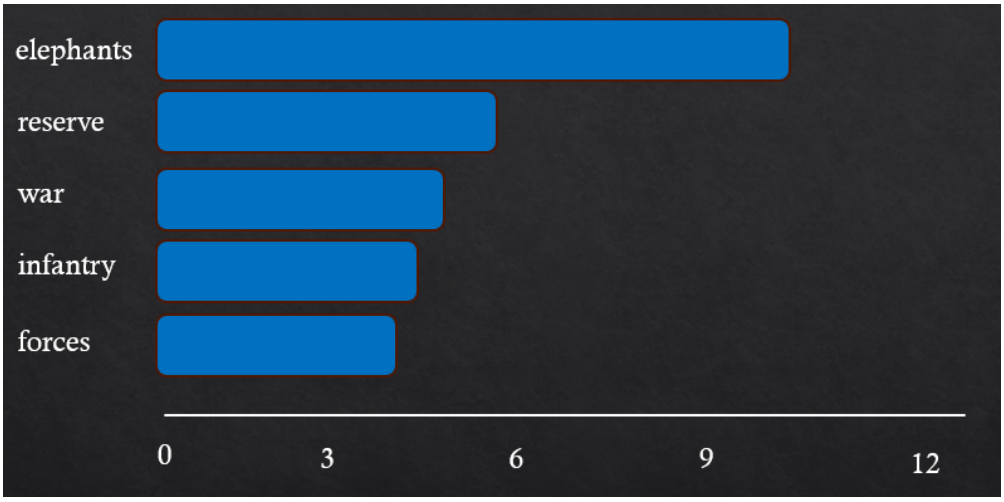


Figure 3.10: Individual query-term contributions to the BM25 relevance score of a retrieved document, illustrating the relative importance of different terms in lexical retrieval.

Table 3.5: BM25 term-level contributions and matching evidence

Query term	Term frequency	IDF	BM25 contribution	Example matching evidence
`elephants`	17	4.473	**10.197**	`war elephants`; use of armored `war elephants`; elephant assault
`reserve`	12	2.440	**5.364**	`reserve counter-attack to halt elephant assault`; reserve deployment
`war`	15	2.062	**4.646**	`war elephants` in participants and military facets
`infantry`	10	2.062	**4.428**	`matchlock infantry advance`; infantry and artillery response
`forces`	6	2.062	**4.047**	Reserve forces decisive in turning the battle

Since both approaches capture different relevance signals, it may be beneficial to combine them into a hybrid retrieval procedure that can potentially improve retrieval robustness. For

this reason, we additionally implement a hybrid retrieval pipeline that combines the rankings from both previous retrieval methods.

3.7.3 Hybrid Retrieval

As discussed above, lexical and dense retrieval capture different types of relevance signals. Lexical retrieval is particularly effective at finding exact matches between a query and a battle representation, while dense retrieval can identify semantically related records even when the query and the relevant representation use different wording. This motivates the use of a hybrid retrieval pipeline that combines the rankings from both methods.

The scores produced by BM25 and dense retrieval are not directly comparable because they use different scoring functions and operate on different scales. We therefore combine the two rankings using Reciprocal Rank Fusion (RRF) (Cormack et al., 2009). RRF combines the rankings based on the position of each battle in the individual result lists. For every candidate battle d, its BM25 and dense ranks are converted into reciprocal-rank contributions using $1/(k + \mathrm{rank})$. A single fused score is then obtained using the following formula, as shown in Equation (3.4):

$$\mathrm{RRF}(d) = \sum_{r} \frac{1}{k + \mathrm{rank}_r(d)} \tag{3.4}$$

Here, r denotes the retrieval system and k is a smoothing constant, which was set to 60 in our implementation. A battle that is ranked highly by both lexical and dense retrieval receives two relatively large reciprocal-rank contributions, one from each retrieval system. After an RRF score has been calculated for every candidate battle, the candidates are sorted in descending order of their fused RRF scores to obtain the final hybrid ranking. In our hybrid retrieval pipeline, the top 100 results from BM25 and the top 100 results from dense retrieval are used as the candidate pool. After RRF fusion, only the highest-ranked 50 battles are retained as the final hybrid retrieval results.

As an example, let us take the query *Find a major battle fought in BCE during the Second Punic War in which Carthage fought Rome*. Lexical retrieval ranks the Battle of Zama (Q200056) 23rd, while dense retrieval ranks this battle 5th. The fused score is calculated as $\frac{1}{60+23} + \frac{1}{60+5} \approx 0.02743$. The same RRF calculation is applied to every unique candidate battle appearing in the top 100 BM25 results or the top 100 dense-retrieval results. After sorting the obtained results by decreasing RRF score, 10 battles receive higher fused scores than the Battle of Zama, so the Battle of Zama is ranked 11th in the hybrid pipeline.

3.8 Answer Generation

According to RQ3, we aim to identify how does the use of structured battle representations as external evidence affect LLM-based question-answering quality compared with direct LLM querying and raw-text augmentation? For this purpose, after retrieval, the final stage of the pipeline generates the set of battles that satisfy a query and compares three answer-generation configurations: a baseline LLM without retrieved context, an LLM augmented with structured JSON battle records, and an LLM augmented with the corresponding raw Wikipedia texts. The same underlying language model and generation configuration are

used across all three settings. The information provided to the LLM differs depending on the configuration.

In addition, we conduct two types of generation experiments. Oracle-context generation evaluates generation quality in isolation from retrieval, where the model is provided with a predefined list of candidate battles that includes gold battles and negative examples. The end-to-end RAG experiment evaluates generation together with retrieval; instead of receiving a predefined candidate set, the model uses the battles returned by the hybrid retrieval pipeline. Table 3.6 shows the comparison between 2 generation experiments in terms of their goal, provided context, exposition of golden set, and use of retrieval pipeline.

Table 3.6: Comparison of oracle-context and end-to-end experiments

	Oracle-context experiment	End-to-end experiment
Candidate battles	Gold + predefined negatives	Returned by hybrid retrieval
Gold present in context?	Yes	No
Baseline receives candidates?	Yes	No
JSON context	Candidate JSON records	Retrieved JSON records
Raw-text context	Candidate article texts	Retrieved article texts
Main question	Can the LLM select the right battles given correct candidates?	Can the complete retrieval and generation pipeline answer correctly?
Retrieval errors included?	No	Yes

3.8.1 Oracle-context generation

In the Oracle-context generation experiment, our goal is to evaluate the ability of the model to choose the correct set of battles from a provided list of candidate battles that contains both gold battles and closed-set negatives. All three configurations, including the baseline LLM, JSON-augmented LLM, and raw-text-augmented LLM, receive the same set of candidate battles in the context. Figure 3.11 shows how the process works for the baseline configuration. The model receives only the instruction to answer the question using its internal knowledge and select relevant battles from the candidate pool.

As shown in Figure 3.12, the JSON-augmented configuration additionally receives structured battle representations as context. These structured representations include battle records from both the gold set and the closed-set negatives. The instruction is similar: the model must choose relevant battles from the provided candidate list, but this time it is prohibited from using its internal knowledge and must answer only using the provided context. This constraint is introduced to isolate the contribution of external context and determine whether the supplied battle representations contain sufficient information to produce the correct set of relevant battles. Without this constraint, it would be difficult to determine whether the model's answer was based on the provided context or on its parametric memory. The same evidence-use restriction is applied to the raw-text-augmented configuration to enable

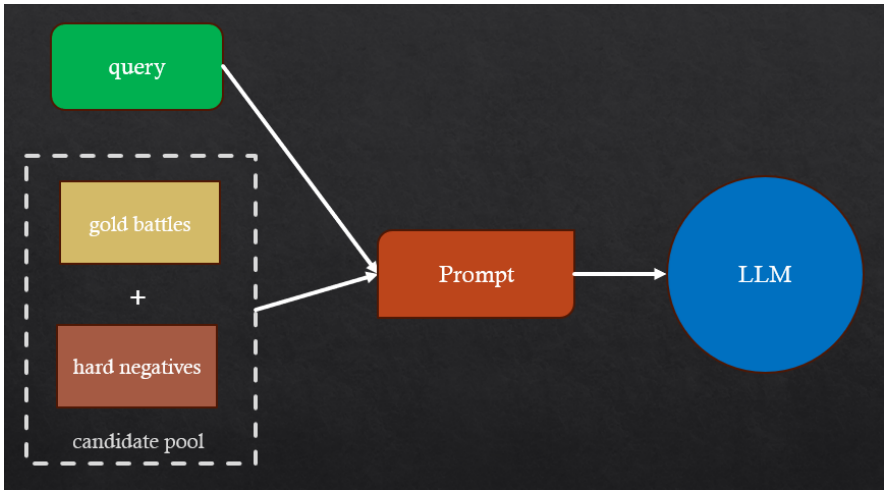


Figure 3.11: Closed-set baseline configuration.

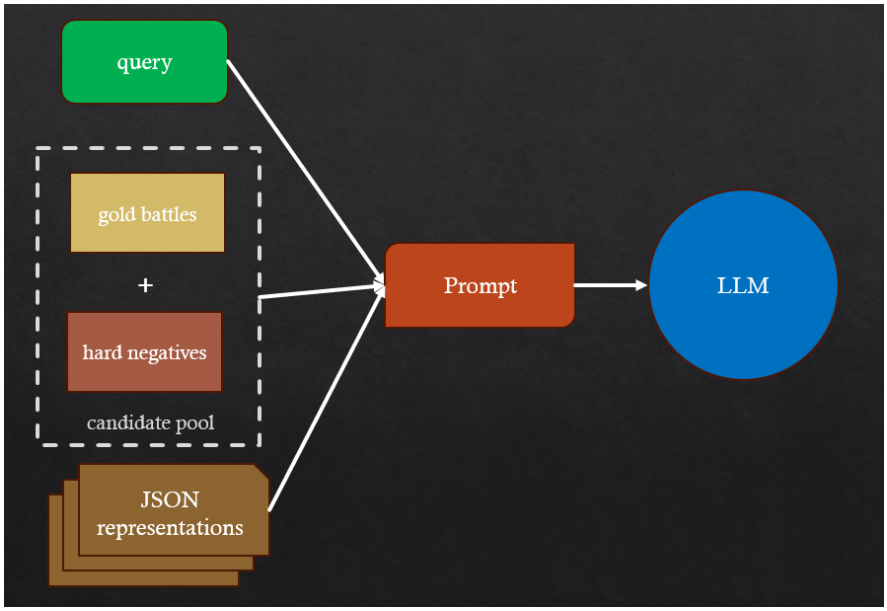


Figure 3.12: Closed-set JSON-augmented configuration.

a controlled comparison. A similar pipeline for the raw-text-augmented configuration is illustrated in Figure 3.13.

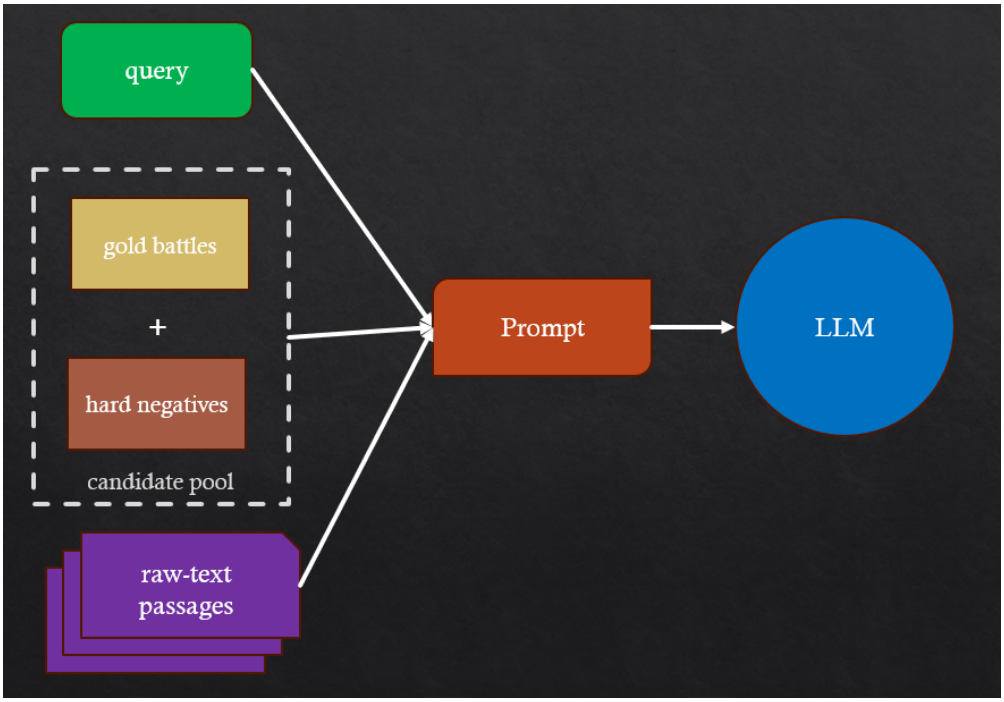


Figure 3.13: Closed-set raw-text-augmented configuration.

3.8.2 End-to-end generation

In the end-to-end RAG experiment, our goal is to evaluate the complete retrieval and answer-generation pipeline under realistic conditions, where the relevant battles are not guaranteed to be present in the model context. Unlike the oracle-context experiment, the baseline LLM does not receive a predefined candidate set. Instead, it receives only the original query and must identify matching battles using its internal knowledge. The JSON-augmented and raw-text-augmented configurations, in contrast, receive battle records selected by the hybrid retrieval pipeline. For each query, the top-k retrieved battles are used as context, so the final answer depends on both retrieval quality and the model's ability to correctly interpret the retrieved evidence. All three configurations are shown in Figure 3.14, Figure 3.15 and Figure 3.16.

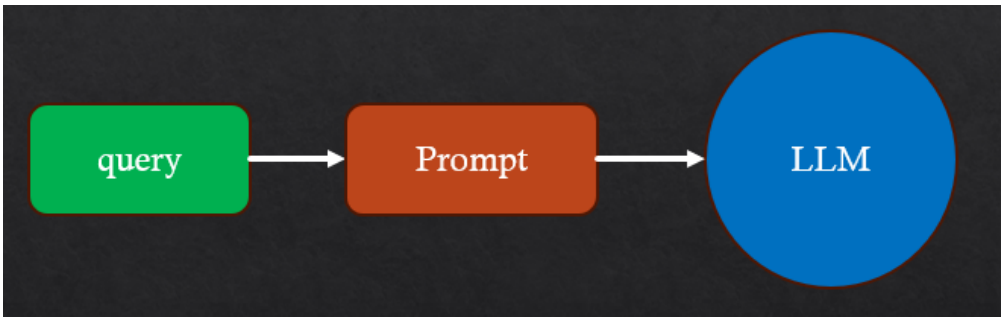


Figure 3.14: End-to-end baseline model configuration.

In the JSON-augmented configuration, the model receives the generation query and the

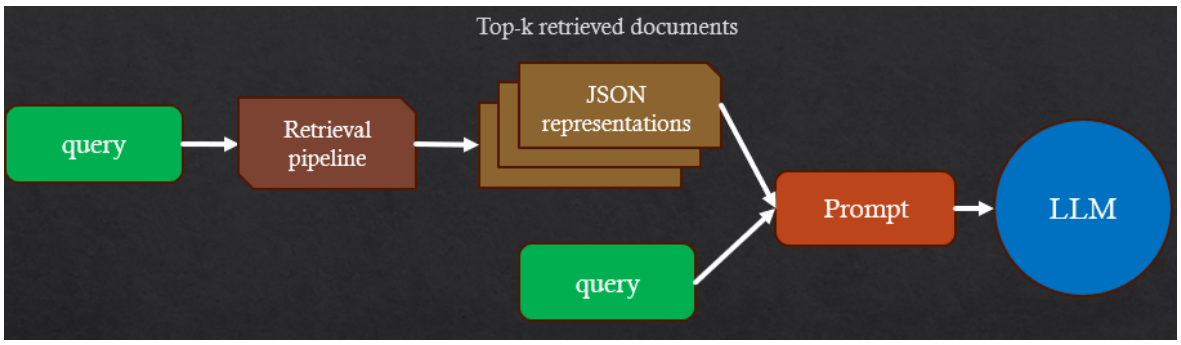


Figure 3.15: End-to-end JSON-augmented model configuration.

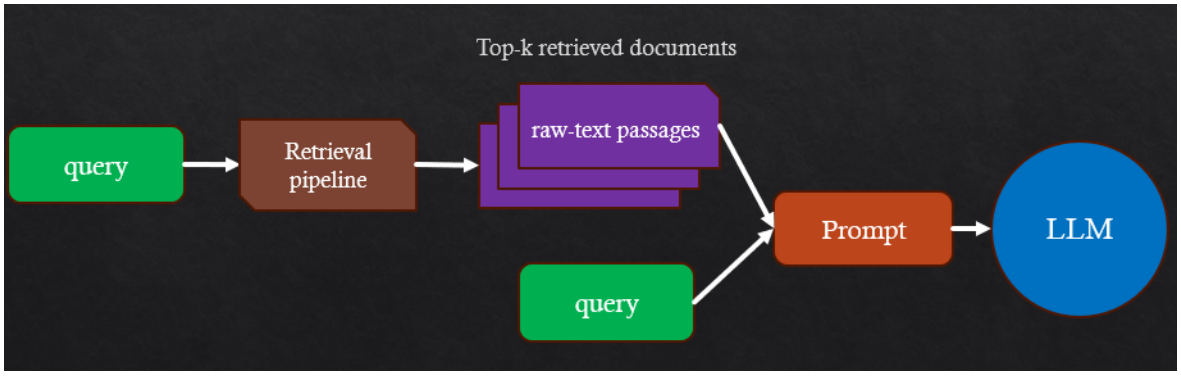


Figure 3.16: End-to-end raw-text-augmented model configuration.

top-k structured battle passages retrieved by the hybrid retrieval pipeline. The model is instructed to use the battle records as evidence and not to rely on its internal knowledge. It may return only battles that occur in the retrieved context and are directly supported by it. The raw-text-augmented configuration follows the same principle, but the retrieved context consists of the corresponding Wikipedia article text instead of structured battle representations. For the latter two configurations, the same restriction against using the model's internal knowledge is applied in order to compare the augmented configurations under equivalent evidence conditions.

The end-to-end setup is designed to evaluate the full RAG process, including both retrieval and generation, and to analyze how the model performs across all three configurations without access to a predefined candidate battle pool.

In the next chapter, we discuss the experimental setup in detail, including the set of testing queries, evaluation metrics, and experiment design.

4 Experimental Design

This chapter presents the experimental design used to evaluate the main components of our system. We first describe the construction of our query set, including query types, difficulty levels, context rarity, gold battles, and closed-set negatives. We then present three groups of experiments: the Oracle-context generation experiment for evaluating generation in isolation, the retrieval experiment for comparing lexical, dense, and hybrid retrieval, and the end-to-end RAG experiment for evaluating the complete retrieval and generation pipeline under more realistic conditions.

4.1 Query Set

To prepare the query set, we first designed a JSON structure for each query object so that it could be processed during the experimental stage and later results analysis.

```
{
  "query_id": "q001",
  "query": "Find major battles fought in 1066 during the struggle for the English throne.",
  "generation_query": "...",
  "query_type": "Temporal",
  "query_structure": "Conjunctive",
  "gold_battles": {
    "Q1113659": "Battle of Fulford",
    "Q203225": "Battle of Stamford Bridge",
    "Q83224": "Battle of Hastings"
  },
  "closed_set_negatives": {
    "Q222013": "Battle of Bosworth Field",
    "Q188495": "Battle of Agincourt",
    "Q27759": "Battle of Crecy",
    "Q201692": "Battle of Poitiers",
    "Q499626": "Battle of Stirling Bridge",
    "Q4590555": "Battle of Loudoun Hill"
  },
  "explanation": "...",
  "query_difficulty": "easy",
  "context_rarity": "common"
}
```

Listing 4.1: Illustrative evaluation query JSON object.

Listing 4.1 shows a complete example of a query JSON object, including all relevant fields such

as `query_id`, `query`, `query_type`, `gold_battles`, `closed_set_negatives`, and `context_rarity`. Each query is identified by `query_id`. The question provided to the LLM is contained in the `query` field. Another field, `generation_query`, is used as input to the LLM in oracle-context experiments, where the model is additionally aware of the set of candidate battles from which it must choose. The `query_type` field defines the query category. There are six main query types: Temporal, Participant-related, Spatial/Terrain, Weapon/Unit-related, Outcome-related, and Narrative/Tactical, as well as one additional category, Similarity, which is tested separately in some experiments. Table 4.1 provides additional information about the query types, including a description of each category, the number of questions used, and an example query. In total, there are 102 queries: 92 from the six main query types and 10 from the Similarity category. The `gold_battles` field contains a JSON object that

Table 4.1: Overview of evaluation query types

Query type	Description	Count	Example query
Temporal	Queries involving years, historical periods, campaigns, or chronological relationships.	10	*Find major battles fought in 1066 during the struggle for the English throne.*
Spatial / Terrain	Queries involving geographical locations, terrain features, and battlefield conditions.	13	*Find battles where a smaller defending force exploited a narrow or restricted approach to reduce the attacking army's numerical advantage.*
Participant-related	Queries concerning commanders, armies, alliances, and relationships between forces.	15	*Find major battles in which Hannibal personally commanded Carthaginian forces against Roman armies.*
Outcome-related	Queries focusing on battle results, defeats, retreats, and strategic consequences.	12	*Find battles where the losing army was surrounded or trapped and then suffered catastrophic destruction rather than making an orderly withdrawal.*
Weapon / Unit-related	Queries concerning weapons, military units, technologies, and tactical roles.	20	*Find battles where war elephants were deployed prominently but failed to secure victory for the army using them.*
Narrative / Tactical	Queries describing tactical patterns, event sequences, and battlefield maneuvers.	22	*Find battles where a successful encirclement trapped a large enemy force and caused catastrophic defeat.*
Similarity	Queries comparing battles based on tactical, operational, terrain, or outcome characteristics.	10	*Find battles similar to Cannae in which encirclement or converging attacks trapped a large enemy force and led to catastrophic defeat.*
Total		102	

maps each battle's Wikidata ID to its corresponding name. As illustrated in Listing 4.1, the golden battles for the query *Find major battles fought in 1066 during the struggle for the English throne.* are Battle of Fulford(`Q1113659`), Battle of Stamford Bridge(`Q203225`), Battle of Hastings(`Q83224`). The `closed_set_negatives` field is used mainly for oracle-context generation experiment to form a battle set given to the model that contains both `golden_battles` and similar negative battles aimed to confuse the model. In the next field `query_difficulty` we define 3 difficulty levels such as easy, medium, hard based on the amount of reasoning required. Easy queries are based on explicit factual clues, medium require knowledge of several attributes or interpreting a tactical or causal pattern, while hard queries need more complex semantic reasoning over longer event sequences or abstract similarities between battles. Queries were also characterized by `context_rarity`. This field reflects how common or obscure the referenced battle and the related information are likely to be in general knowledge. The motivation was to reduce the advantage of a baseline LLM, which might easily answer questions about well-known battles from its parametric memory but may have more difficulty answering questions about rare battles. Rare-context queries therefore focus

on less famous battles.

The complete benchmark of 102 queries was constructed with the assistance of the GPT-5.6 Sol model using a corpus-grounded generation process. We did not ask the model to generate queries based on its own historical knowledge. Instead, information about historical battles was provided in smaller batches of around 1,000 battle records at a time. These records represented compact battle structures capturing the most important factual and narrative information needed for the model to reason over them and use this information for query generation. They were constructed based on our complete JSON battle knowledge base, KB2.

For each batch, GPT-5.6 Sol was instructed to identify meaningful factual and narrative properties, shared patterns, and formulate query objects according to the predefined structure shown in Listing 4.1. The model was additionally instructed to generate queries of different types described above and to include both common and rare battles. In the output, the model also proposed a gold set containing Wikidata QIDs and names of relevant battles, as well as a set of closed-set negatives. For hard negatives, the model selected battles that were similar to the gold battles but failed one or several essential conditions of the query. An explanation field was also added, where the model briefly described why the gold-set battles were selected. Overall, the query-set generation process is illustrated in Figure 4.1.

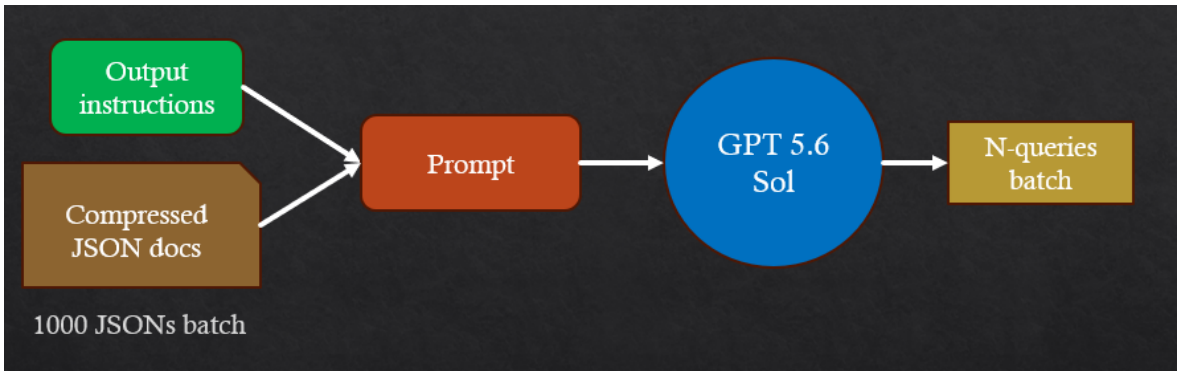


Figure 4.1: Evaluation query set generation pipeline.

4.2 Oracle-context Generation Experiment

As described in Section 3.8.1, the Oracle-context generation experiment aims to evaluate the ability of the model to choose the correct set of battles from a provided list of candidate battles consisting of both gold battles and closed-set negatives. The process for all three LLM configurations including the baseline LLM, JSON-augmented LLM, and raw-text-augmented LLM was illustrated in detail in Figure 3.11, Figure 3.12 and Figure 3.13. For all experiments, GPT-OSS-120B is used again. In this experiment, there is no retrieval step, so we evaluate generation performance in isolation. The baseline LLM does not receive any external context and performs reasoning using only its internal knowledge. In the JSON-augmented and raw-text-augmented configurations, the LLM receives either JSON battle representations or raw-text Wikipedia articles, respectively. The provided battle instances include both battles from the gold set and battles from the closed-set negative set. We use KB2 as the battle knowledge corpus from which battle representations are provided to the JSON-augmented model. For additional analysis, we also include results from a JSON-augmented model that received battle representations from KB1 to evaluate how the LLM Verification and Regeneration step improved the structured battle knowledge base.

To gain more insight into generation performance and the differences between the three configurations, we divide the experiment into three parts. In the first part, we analyze the overall generation performance and compare the results obtained using KB1 and KB2. In the second part, we analyze generation performance for common and rare battles separately, by filtering the initial query set. In the third part, we evaluate the LLM configurations across different query types to examine how query type affects the ability of the LLM to provide accurate answers. Similarity queries are excluded from this overall comparison and evaluated separately. Table 4.2 illustrates all three parts of the experiment, including their aims, testing subjects, and descriptions. To evaluate generation performance, we use four main metrics:

Table 4.2: Overview of the generation evaluation experiments

Part	Aim	Testing subjects	Description
1. Overall	Compare overall generation performance across model configurations.	Baseline; Raw-text-augmented; JSON-augmented (KB2); JSON-augmented (KB1)	Evaluated across the 92 main queries using generation metrics. KB1 is included to assess the effect of the Verification and Regeneration step.
2. Common vs. Rare	Analyze how battle rarity affects generation performance.	Common queries; Rare queries	Performance is compared separately for common and rare battle queries.
3. Query Type	Compare generation performance across semantic query categories.	Temporal; Participant-related; Spatial/Terrain; Weapon/Unit-related; Outcome-related; Narrative/Tactical; Similarity	Performance is analyzed separately for each query type. Similarity is evaluated on its separate 10-query set.

precision, recall, F1, and exact match. The metrics are computed independently for each query and then macro-averaged across all evaluated queries. Additionally, we count the number of parse errors and run errors.

The model is instructed to follow a specific JSON output format, which is parsed after generation. The returned battle QIDs are used for quantitative evaluation. Duplicate predicted QIDs are removed before the metrics are calculated. For each query, precision and recall are computed by comparing the predicted QID set with the corresponding gold QID set, while F1 is calculated as their harmonic mean. Exact match is assigned a value of 1 only if the predicted set of battles is identical to the complete gold set, and 0 otherwise. We use the `battle_names` and `explanations` fields for debugging purposes and overall analysis. Responses that cannot be parsed into the expected JSON structure or that contain invalid QIDs are recorded as parse errors and receive zero scores for the generation metrics. If the model encounters a failure during execution, we record it separately as a run error.

4.3 Retrieval Experiment

The methodology and implementation of the three retrieval pipelines, lexical, dense, and hybrid retrieval, were previously discussed in Section 3.7. In this section, we discuss the

retrieval experiments used to compare these approaches. The retrieval experiment aims to evaluate the retrieval step in isolation and determine how well each retrieval algorithm ranks battle instances from our historical battle knowledge base for a given query. All three strategies were evaluated using the same set of 102 raw-text queries and compared against the corresponding gold sets of relevant historical battles. For each query, the BM25 and dense retrieval algorithms independently returned the top 100 candidate battles, ranked according to their retrieval scores. The two ranked lists were then combined using Reciprocal Rank Fusion, producing a hybrid ranking that was limited to the top 50 battles.

We used six metrics to evaluate the performance of each retrieval algorithm: Recall@10, Recall@20, Recall@50, Mean Reciprocal Rank (MRR), Mean Average Precision at 10 (MAP@10), and normalized Discounted Cumulative Gain at 10 (nDCG@10). Recall@k measures the proportion of relevant battle records retrieved among the top-k results. MRR evaluates the rank position of the first relevant battle. MAP@10 assesses how highly relevant battle records are ranked among the top 10 candidates. Finally, nDCG@10 evaluates ranking quality by assigning higher importance to relevant battles appearing at earlier positions. A detailed description and mathematical formulation of the metrics is provided in Table 4.3. Binary relevance labels were used: battles included in the query-specific gold set were treated as relevant, while all other battles were treated as non-relevant. All metrics were computed independently for each query and then macro-averaged across the query set, giving each query equal weight. The experiments were again divided into four parts, as

Table 4.3: Retrieval evaluation metrics and their interpretations

Metric	Formula	Interpretation
Recall@k	$\mathrm{Recall@k} = \frac{\lvert R_k \cap G \rvert}{\lvert G \rvert}$	Measures the proportion of relevant battles retrieved within the top k results.
MRR	$MRR = \frac{1}{N}\sum_{i=1}^{N}\frac{1}{rank_i}$	Measures how early the first relevant battle appears in the ranking.
MAP@10	$MAP@10 = \frac{1}{N}\sum_{i=1}^{N} AP_i@10$	Measures how highly and consistently relevant battles are ranked within the top 10 results.
nDCG@10	$nDCG@10 = \frac{DCG@10}{IDCG@10}$	Measures ranking quality while giving greater importance to relevant battles appearing at higher ranks.

shown in Table 4.4. First, we report the overall retrieval performance across the complete query set and compare the results obtained using the KB1 and KB2 battle corpora. In the second experiment, we analyze retrieval quality separately for queries describing common and rare battles. Finally, the queries are filtered by `query_type`, and retrieval performance is evaluated separately across the different query types. Listing 4.2 shows an example of the JSON output produced by the retrieval pipeline when it was run on a set of rare queries.

Table 4.4: Overview of the retrieval evaluation experiments

Part	Aim	Testing subjects	Description
1. Overall Performance	Compare the overall retrieval performance of the three retrieval algorithms.	BM25; Dense; Hybrid	Performance is evaluated across the complete query set using Recall@10, Recall@20, Recall@50, MRR, MAP@10, and nDCG@10.
2. Common vs. Rare	Analyze how battle rarity affects retrieval quality.	Common queries; Rare queries	Retrieval performance is compared separately for queries concerning common and rare battles.
3. Query Type	Compare retrieval performance across different semantic query categories.	Temporal; Participant-related; Spatial/Terrain; Weapon/Unit-related; Outcome-related; Narrative/Tactical; Similarity	Retrieval metrics are analyzed separately for each query type to identify differences between BM25, dense, and hybrid retrieval.


```
{
  "bm25": {
    "Recall@10": 0.508,
    "Recall@20": 0.586,
    "Recall@50": 0.651,
    "MRR": 0.543,
    "MAP@10": 0.428,
    "nDCG@10": 0.475
  },
  "dense": {
    "Recall@10": 0.505,
    "Recall@20": 0.565,
    "Recall@50": 0.596,
    "MRR": 0.521,
    "MAP@10": 0.425,
    "nDCG@10": 0.468
  },
  "hybrid": {
    "Recall@10": 0.583,
    "Recall@20": 0.615,
    "Recall@50": 0.727,
    "MRR": 0.572,
    "MAP@10": 0.470,
    "nDCG@10": 0.526
  }
}
```


Listing 4.2: Example of the JSON output containing retrieval evaluation metrics for BM25, dense, and hybrid retrieval strategies on rare queries. Metric values are rounded to three decimal places.

4.4 End-to-end RAG Experiment

Following the description of the generation experiments in Section 3.8.2, the end-to-end RAG experiment evaluates the complete pipeline, including both retrieval and answer-generation steps. Unlike the Oracle-context experiment, the set of candidate battles to choose from is not provided in advance. Instead, the retrieval pipeline first selects the top-k battle records from the knowledge base, and these retrieved records are then passed to the generation stage. We use the hybrid retrieval pipeline for the retrieval step. We again compare three LLM configurations: the baseline LLM, the raw-text-augmented LLM, and the JSON-augmented LLM. The baseline LLM receives only the original query and relies on its internal knowledge, while the JSON-augmented and raw-text-augmented configurations receive retrieved JSON battle representations or the corresponding raw Wikipedia texts as external context. We additionally analyze the performance of the augmented models for different values of k, representing different numbers of retrieved battle instances. The final performance in this experiment depends on both retrieval quality and the model's ability to correctly interpret the retrieved evidence.

The end-to-end RAG experiment is divided into three parts to test different aspects of the complete pipeline across different query categories and configurations. In the first part, we test the effect of different values of k, representing different numbers of retrieved battle instances provided by the hybrid retrieval algorithm. By default, it returns a ranked list of the top 50 battle records for each query. However, providing 50 JSON representations to the LLM may introduce a substantial amount of irrelevant information and increase the context length. Previous work has shown that LLMs may struggle to effectively utilize relevant information in long contexts, particularly when it appears in the middle of the input prompt (Liu et al., 2024). Therefore, we investigate different k-values representing the top-ranked battles supplied to the LLM. The goal is to find an optimal retrieval depth that balances the presence of relevant battles with answer accuracy.

We use the same set of metrics as in the Oracle-context experiment, including precision, recall, F1, exact match, and error rate. To complement the retrieval quality analysis, we additionally adopt two evaluation metrics inspired by the RAGAS framework: faithfulness and context relevance (Es et al., 2024). Faithfulness measures how well the statements generated by the LLM are supported by the retrieved context, while context relevance evaluates how much of the retrieved information is relevant to answering the original query. These additional metrics help assess the quality of the context provided to the LLM and its usefulness during answer generation.

In our implementation, the metrics are computed in the following way. For each predicted battle, the generated explanation is compared against the corresponding JSON battle representation, and the LLM evaluator marks it as supported only if all factual claims can be inferred from the JSON file. The faithfulness score is then computed as the proportion of fully supported explanations. To evaluate context relevance, each JSON battle representation supplied to the generation model is independently judged as relevant or irrelevant to the query. The context relevance score is computed as the proportion of relevant JSON representations that are useful for answering the question. For both metrics, the LLM evaluator receives only the provided query, the generated explanation, and the retrieved JSON context.

In our experiments, we investigated how different values of k affect the performance and error rate of the augmented models. In addition, we evaluated model performance separately for queries involving common and rare battles and across different query types.

5 Experimental Results

In this chapter we discuss Experiment Results, specifically for Retrieval, Oracle-context generation and End-to-end RAG experiments.

5.1 Retrieval Results

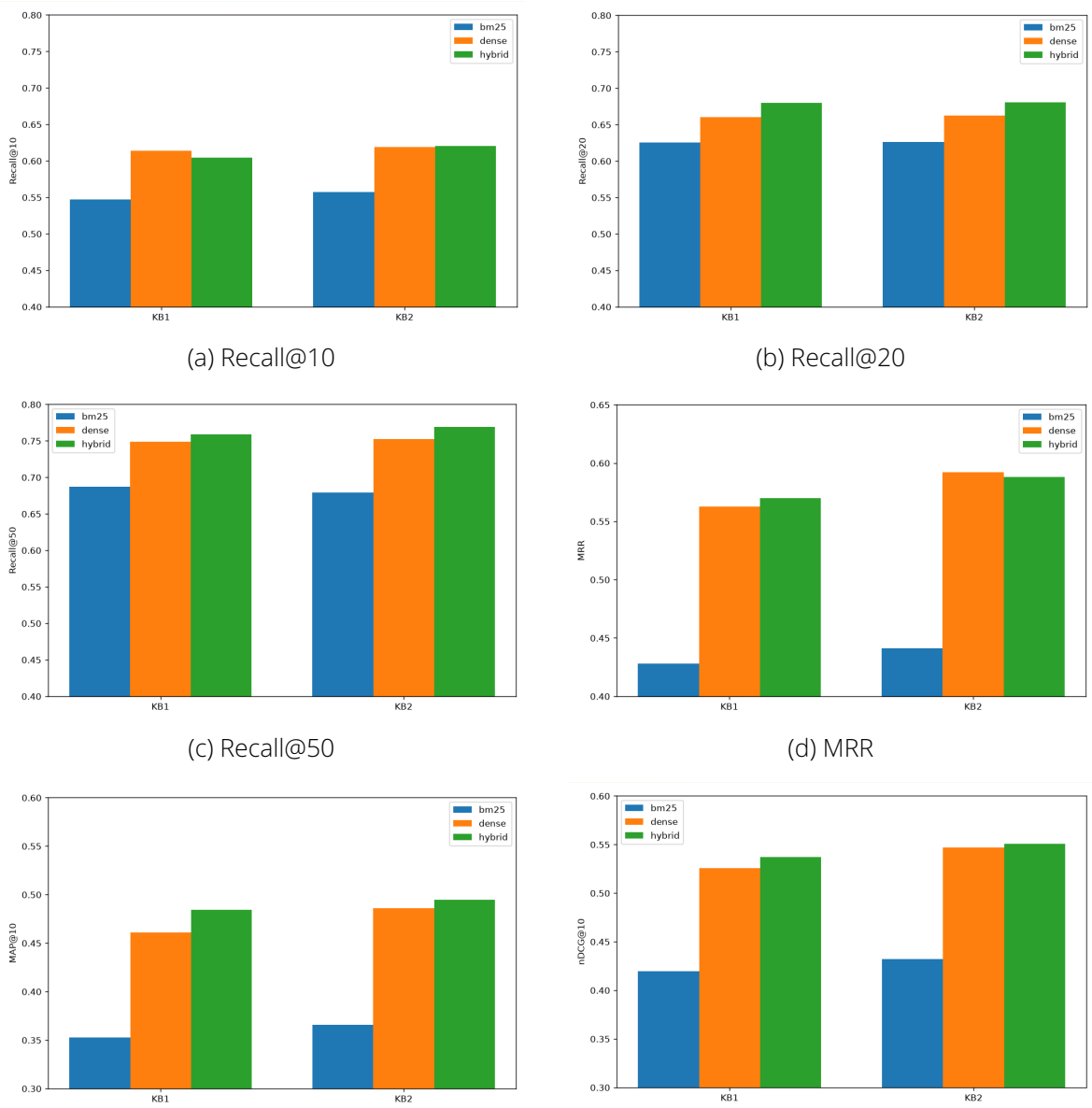
As previously discussed in Chapter 4.3, we conducted three sub-experiments to evaluate different aspects of retrieval quality and performance. First, we report the overall retrieval results across the complete set of 102 queries and compare retrieval performance when using the KB1 and KB2 battle corpora. Second, we focus on queries involving common and rare battles and analyze how context rarity affects the performance of each retrieval strategy. Finally, we investigate how retrieval performance differs across different query types.

5.1.1 Overall Retrieval Performance

Table 5.1 shows the overall retrieval performance across all 102 queries. Additionally, we compare retrieval performance separately using the KB1 and KB2 battle corpora. Figure 5.1 shows Recall@10/20/50, MRR, MAP@10 and nDCG@10 scores for the three retrieval pipelines: lexical, dense, and hybrid retrieval. Additional labels distinguish the results obtained using KB1 (bars on the left) from those obtained using KB2 (bars on the right). Recall scores remain similar across the two knowledge base versions, suggesting that KB2 does not provide a substantial retrieval advantage over KB1. Dense retrieval improves Recall over BM25 by approximately 9.0% on average, while hybrid retrieval further improves Recall over dense retrieval by approximately 1.4%. The higher recall of dense retrieval is consistent with its ability to capture semantic similarity beyond exact lexical overlap.

For KB1, hybrid retrieval outperformed dense retrieval by approximately 1.9% on average across the six evaluation metrics, achieving higher scores in five out of six metrics. Compared with BM25, hybrid retrieval demonstrated an average relative improvement of 21.4%, outperforming it across all six metrics. Similarly, for KB2, hybrid retrieval outperformed dense retrieval by approximately 1.2% on average across five out of six metrics, while achieving an average relative improvement of 21.5% over BM25 across all six metrics.

Overall, these results suggest that the transition from KB1 to KB2 has only a limited effect on retrieval performance, since the relative differences between the retrieval methods remain



(a) Recall@10

(b) Recall@20

(c) Recall@50

(d) MRR

(e) MAP@10

(f) nDCG@10

Figure 5.1: Overall comparison of BM25, dense, and hybrid retrieval performance across KB1 and KB2 using six evaluation metrics: Recall@10, Recall@20, Recall@50, MRR, MAP@10, and nDCG@10.

Table 5.1: Overall retrieval performance of BM25, dense, and hybrid retrieval using KB1 and KB2

	KB1			KB2		
Metric	BM25	Dense	Hybrid	BM25	Dense	Hybrid
Recall@10	0.547	**0.614**	0.605	0.557	0.619	**0.620**
Recall@20	0.625	0.660	**0.680**	0.626	0.663	**0.681**
Recall@50	0.687	0.749	**0.759**	0.679	0.753	**0.769**
MRR	0.428	0.563	**0.570**	0.441	**0.592**	0.588
MAP@10	0.353	0.461	**0.484**	0.366	0.486	**0.495**
nDCG@10	0.419	0.526	**0.537**	0.432	0.547	**0.551**

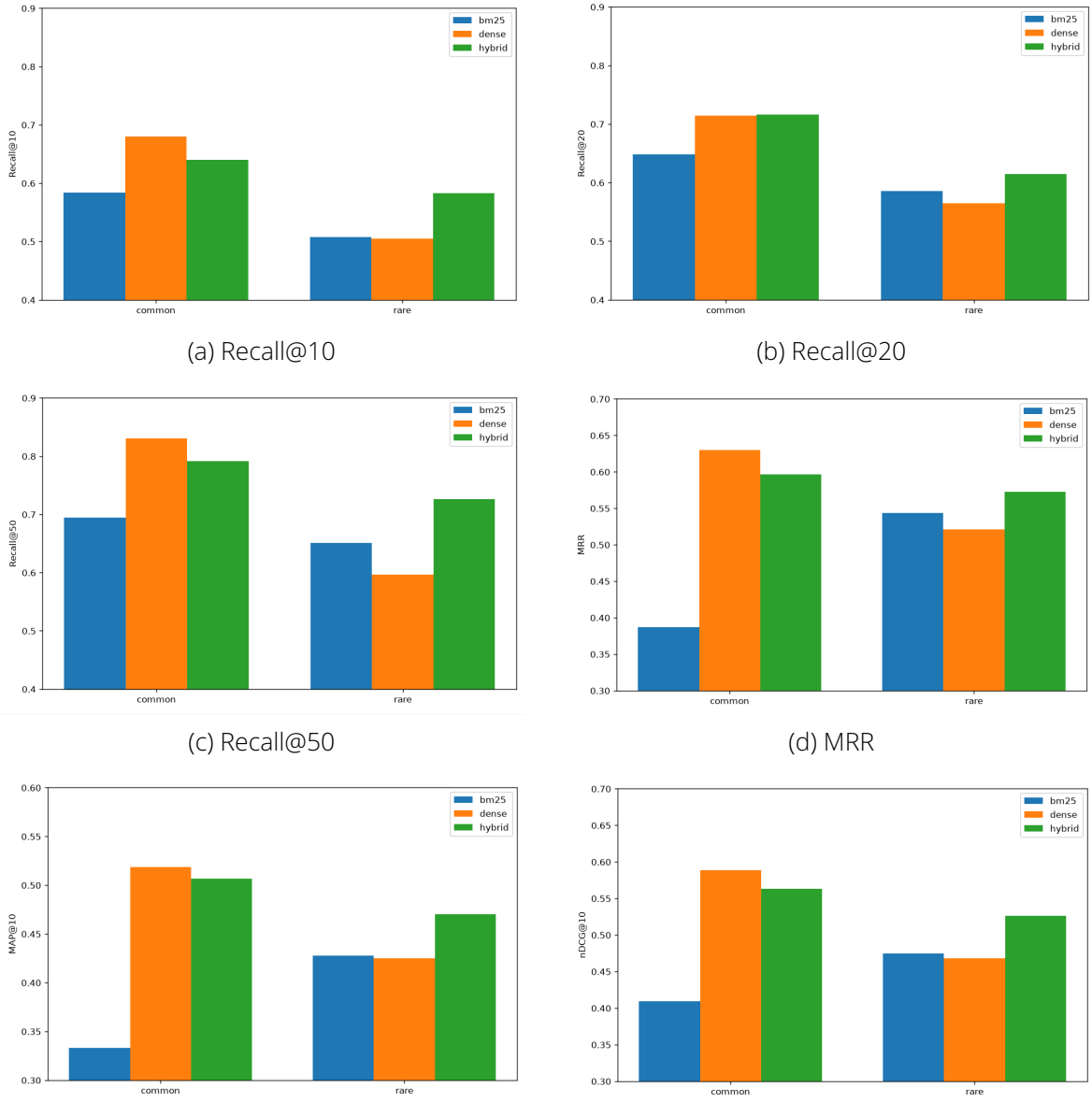
almost unchanged across both knowledge base versions. In both cases, hybrid retrieval consistently performs best or close to best and provides a substantial advantage over BM25, while its improvement over dense retrieval remains relatively small. This indicates that the retrieval method has a stronger influence on retrieval quality than the choice between KB1 and KB2.

5.1.2 Retrieval Performance for Common and Rare Battles

In the second retrieval experiment, we investigated the performance of the retrieval algorithms on queries filtered by common and rare battles. The relative performance changed slightly. We noticed that for common battles, dense retrieval achieves the best results on five of the six evaluation metrics, while for rare-battle queries, hybrid retrieval outperformed dense retrieval across all six evaluation metrics, with an average relative improvement of approximately 13.1%. The largest improvement was observed for Recall@50, where hybrid retrieval achieved a 21.8% higher score. Table 5.2 provides the corresponding metric values for both common- and rare-battle queries. Figure 5.2 additionally visualizes these results. Each bar chart demonstrates the results for one particular metric. For common battles, dense retrieval outperforms hybrid retrieval by around 3.9% on average across all six metrics, with the largest improvement observed for Recall@10 at 6.2%. Compared with BM25, dense retrieval improves the average score by approximately 34.8%, with the largest improvements observed in MRR and MAP@10.

One notable observation is the relatively low performance of BM25 for common battles. Although these battles are generally well known, lexical retrieval still depends on direct term overlap between the query and the JSON battle representation. We therefore decided to inspect several retrieval results in detail and found that semantically equivalent tactical descriptions can still use considerably different wording.

A representative example of this difference is query q048: *Find a medieval battle where English longbows and defensive positioning combined to defeat French mounted and dismounted attacks.* The Battle of Agincourt strongly satisfies these conditions; however, BM25 ranked it only 27th, while dense retrieval placed it in the top three. The reason is that terms in the query such as *defensive positioning*, *mounted attacks*, and *dismounted attacks* are expressed differently in the JSON battle representation, particularly through phrases such as *defensive deployment behind stakes*, *French cavalry charge*, *infantry assault*, and *French men-at-arms*. Dense retrieval was better able to capture these semantic associations. Table 5.3 shows the different wording



(a) Recall@10

(b) Recall@20

(c) Recall@50

(d) MRR

(e) MAP@10

(f) nDCG@10

Figure 5.2: Comparison of BM25, dense, and hybrid retrieval performance for common and rare historical battle queries across six evaluation metrics: Recall@10, Recall@20, Recall@50, MRR, MAP@10, and nDCG@10.

Table 5.2: Retrieval performance of BM25, dense, and hybrid retrieval for common and rare battle queries

	Common			Rare		
Metric	BM25	Dense	Hybrid	BM25	Dense	Hybrid
Recall@10	0.584	**0.680**	0.640	0.508	0.505	**0.583**
Recall@20	0.648	0.715	**0.716**	0.586	0.565	**0.615**
Recall@50	0.694	**0.830**	0.792	0.651	0.596	**0.727**
MRR	0.387	**0.630**	0.596	0.543	0.521	**0.572**
MAP@10	0.333	**0.519**	0.507	0.428	0.425	**0.470**
nDCG@10	0.409	**0.588**	0.563	0.475	0.468	**0.526**

Note: Bold values indicate the best-performing retrieval method for each metric within the corresponding query rarity group.

used in query q048 and the corresponding battle representation. This example illustrates how lexical retrieval can rank semantically similar battles differently because of differences in wording, while dense retrieval is better able to capture equivalent terminology.

Table 5.3: Comparison of query wording and the structured representation of the Battle of Agincourt

Query wording	Agincourt representation
`longbows`	`longbow`, `longbowmen`, `longbow volleys`
`defensive positioning`	`defensive deployment behind stakes`, `defensive position`
`mounted attacks`	`French cavalry charge`, `heavy cavalry`
`dismounted attacks`	`French men-at-arms`, `infantry assault`, `advanced through mud`
`attacks`	`charge`, `assault`, `advance`

Overall, these results suggest that the relative performance of the retrieval pipelines depends less on context rarity itself and more on how the query and the corresponding JSON battle representation are formulated. In particular, retrieval quality is strongly influenced by the amount of lexical overlap between the query and the battle record, as well as by how well their wording expresses the same underlying semantic concepts. Dense retrieval is therefore particularly useful when tactically similar information is described using different terminology. At the same time, hybrid retrieval achieves strong results for both common- and rare-battle queries by combining lexical and semantic relevance signals, and provides a substantial improvement especially for rare-battle queries.

5.1.3 Retrieval Performance by Query Type

In the next sub-experiment, we explored how different query types affect the performance of the retrieval methods. Table 5.4, Table 5.5 and Table 5.6 shows clear differences across

all seven query types. Temporal queries achieve the strongest overall results, with dense retrieval reaching the highest average score across the six metrics at approximately 0.890, compared with 0.802 for lexical retrieval and 0.868 for hybrid retrieval. Dense retrieval improves the average score over BM25 by about 11.1% across the six metrics, while also achieving near-ceiling Recall values, including Recall@20 = 1.0 and Recall@50 = 1.0.

Table 5.4: Retrieval performance across participant-related, spatial/terrain, and temporal query types

	Participant-related			Spatial / Terrain			Temporal		
Metric	BM25	Dense	Hybrid	BM25	Dense	Hybrid	BM25	Dense	Hybrid
Recall@10	0.666	0.770	**0.792**	**0.558**	0.433	0.503	0.800	**0.950**	0.913
Recall@20	0.811	0.826	**0.874**	**0.571**	0.471	0.548	0.925	**1.000**	0.975
Recall@50	**0.930**	0.841	0.926	0.596	0.593	**0.676**	**1.000**	**1.000**	**1.000**
MRR	0.538	**0.733**	0.601	0.364	0.521	**0.637**	0.686	**0.833**	0.777
MAP@10	0.422	**0.599**	0.517	0.291	0.315	**0.472**	0.684	0.740	**0.746**
nDCG@10	0.514	**0.686**	0.621	0.374	0.388	**0.513**	0.716	**0.819**	0.796

Note: Bold values indicate the best-performing retrieval method for each metric within the corresponding query type.

Table 5.5: Retrieval performance of BM25, dense, and hybrid retrieval across outcome-related, narrative/tactical, and weapon/unit-related query types

	Outcome-related			Narrative / Tactical			Weapon / Unit-related		
Metric	BM25	Dense	Hybrid	BM25	Dense	Hybrid	BM25	Dense	Hybrid
Recall@10	0.403	0.413	**0.424**	0.373	**0.548**	0.469	0.650	0.663	**0.713**
Recall@20	0.444	0.429	**0.465**	0.388	**0.578**	0.560	**0.746**	0.729	0.738
Recall@50	0.493	**0.612**	0.499	0.414	**0.753**	0.693	0.788	0.750	**0.842**
MRR	0.179	**0.375**	0.356	0.360	**0.532**	0.457	0.544	0.624	**0.735**
MAP@10	0.163	**0.343**	0.311	0.281	**0.418**	0.344	0.428	0.546	**0.643**
nDCG@10	0.226	**0.368**	0.352	0.319	**0.477**	0.404	0.514	0.594	**0.685**

Note: Bold values indicate the best-performing retrieval method for each metric within the corresponding query type.

Table 5.6: Retrieval performance of BM25, dense, and hybrid retrieval for Similarity queries

	Similarity		
Metric	BM25	Dense	Hybrid
Recall@10	0.050	**0.307**	0.203
Recall@20	0.190	**0.390**	0.303
Recall@50	0.363	0.455	**0.530**
MRR	0.046	**0.204**	0.133
MAP@10	0.010	**0.122**	0.055
nDCG@10	0.024	**0.189**	0.109

Note: Bold values indicate the best-performing retrieval method for each metric.

In contrast, Similarity queries remain the most difficult for lexical retrieval. BM25 achieves particularly low scores, with Recall@10 = 0.05, MRR ≈ 0.046, and MAP@10 = 0.01. Dense retrieval, on the other hand, reaches 0.307, 0.204, and 0.122 for these metrics, respectively. Across all six metrics, dense retrieval improves the average score from approximately 0.114 for BM25 to 0.278, corresponding to a relative improvement of about 143.9%. This provides strong evidence that semantic matching is especially helpful for Similarity queries, where lexical overlap may be insufficient.

We observed a different pattern for Spatial/Terrain queries. Here, BM25 and dense retrieval achieve similar average scores across the six metrics, at approximately 0.459 and 0.453, respectively, while hybrid retrieval achieves a higher average score of approximately 0.558. This corresponds to a 23.1% relative improvement over dense retrieval. The largest gains are observed in the ranking-sensitive metrics: MRR increases from 0.521 for dense retrieval to 0.637 for hybrid retrieval, MAP@10 increases from 0.315 to 0.472, and nDCG@10 increases from 0.388 to 0.513, showing a similar trend.

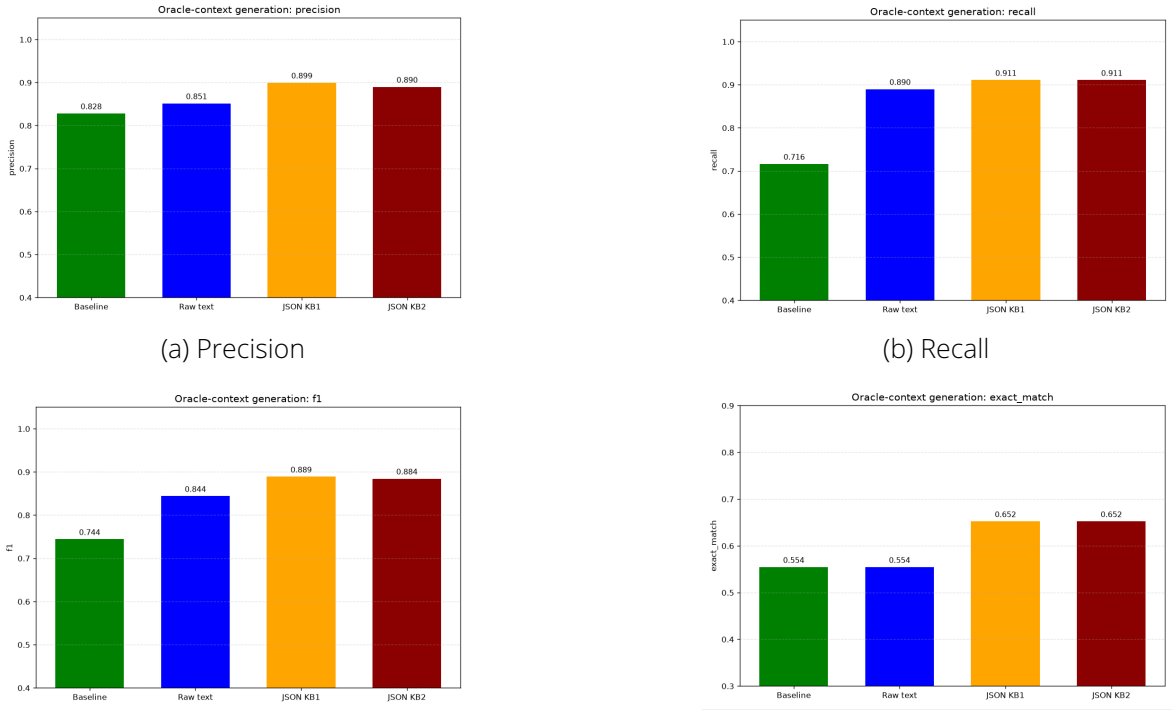
Overall, the hybrid retrieval strategy provides a very consistent improvement over BM25, achieving higher scores in 37 of 42 metric comparisons and higher average performance across all six metrics for all seven query types. However, its advantage over dense retrieval is more selective. Hybrid retrieval achieves better results in 19 of 42 metric comparisons and a higher average score in only two query types: Spatial/Terrain and Weapon/Unit-related. This suggests that although hybrid retrieval provides a robust improvement over purely lexical retrieval, its benefit over dense retrieval depends more strongly on the characteristics of the query type.

5.2 Oracle-context Generation Results

In Chapter 4.2, we introduced the Oracle-context generation experiment, which is designed to evaluate the generation step in isolation, without the influence of retrieval quality. For each query, the model is provided with a predefined set of candidate battles to choose from, containing both battles from the gold set and hard negatives. We divided the Oracle-context generation experiment into three parts. First, we evaluated overall generation performance across the complete query set by comparing the baseline LLM, raw-text-augmented LLM, and JSON-augmented LLM, while also assessing the JSON-augmented results obtained using KB1 and KB2 separately. In the second sub-experiment, we analyzed generation performance separately for queries involving common and rare battles. Finally, we compared generation performance across the different query types. We used four main metrics for the analysis: precision, recall, F1, and exact match. In addition, we recorded the number of parse errors and run errors.

5.2.1 Overall Generation Performance

We summarize the Oracle-context generation results in Table 5.7 and additionally visualize them in Figure 5.3. The comparison between KB1 and KB2 shows that the verification and regeneration pipeline used to construct KB2 did not lead to a substantial improvement over KB1. As shown in Table Table 5.7, KB2 achieved almost identical recall and the same exact-match score as KB1, while precision and F1 were slightly lower. The bar chart further illustrates the small differences between the two JSON-augmented configurations.



(a) Precision

(b) Recall

(c) F1 Score

(d) Exact Match

Figure 5.3: Overall performance comparison of the baseline, raw-text, JSON KB1, and JSON KB2 configurations in the oracle-context generation experiment across four evaluation metrics: Precision, Recall, F1, and Exact Match.

Table 5.7: Overall Oracle-context generation performance across baseline, raw-text-augmented, JSON KB1, and JSON KB2 configurations

Metric	Baseline	Raw-text	JSON KB1	JSON KB2
Precision	0.828	0.851	**0.899**	0.890
Recall	0.716	0.890	**0.911**	**0.911**
F1	0.744	0.844	**0.889**	0.884
Exact Match	0.554	0.554	**0.652**	**0.652**

Note: Bold values indicate the best-performing configuration for each metric.

In contrast, both JSON-augmented configurations clearly outperform the baseline LLM and the raw-text-augmented model across all four evaluation metrics. When using KB1, precision increased from 0.828 to 0.899 compared with the baseline LLM, corresponding to an improvement of 8.6%. Recall, F1, and exact match also improved considerably, by 27.2%, 19.5%, and 17.7%, respectively. The JSON-augmented models also perform slightly better than the raw-text-augmented model. Compared with raw-text augmentation, KB1 increased precision from 0.851 to 0.899, recall from 0.890 to 0.911, and exact match from 0.554 to 0.652, corresponding to improvements of 5.7%, 2.4%, and 17.7%, respectively. On average across the four evaluation metrics, the KB1 JSON-augmented model achieved approximately 17.9% higher performance than the baseline LLM and 6.8% higher performance than the raw-text-augmented model. These results indicate an advantage of the structured battle representation in the controlled Oracle-context experiment.

5.2.2 Generation Performance for Common and Rare Battles

In this sub-experiment, we examine model performance in the Oracle-context setting separately for common- and rare-battle queries. Table 5.8 compares Oracle-context generation performance across common and rare battle queries. shows the metric scores for all four model configurations, while Figure 5.4 visualizes precision, recall, F1, and exact match.

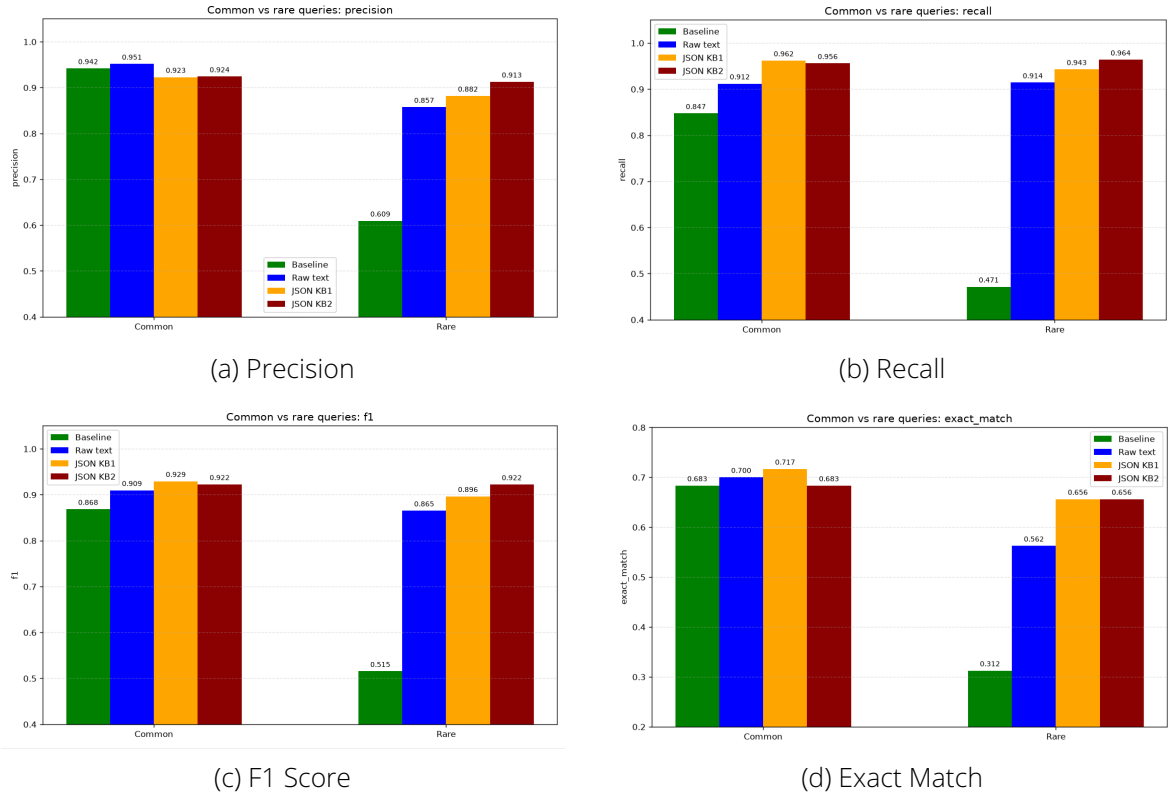


Figure 5.4: Comparison of baseline, raw-text, JSON KB1, and JSON KB2 performance on common and rare historical battle queries in the oracle-context generation experiment, evaluated using Precision, Recall, F1, and Exact Match.

Table 5.8: Oracle-context generation performance for common and rare battle queries across baseline, raw-text-augmented, JSON KB1, and JSON KB2 configurations

	Common				Rare			
Metric	Baseline	Raw-text	JSON KB1	JSON KB2	Baseline	Raw-text	JSON KB1	JSON KB2
Precision	0.942	**0.951**	0.923	0.924	0.609	0.857	0.882	**0.913**
Recall	0.847	0.912	**0.962**	0.956	0.471	0.914	0.943	**0.964**
F1	0.868	0.909	**0.929**	0.922	0.515	0.865	0.896	**0.922**
Exact Match	0.683	0.700	**0.717**	0.683	0.313	0.563	**0.656**	**0.656**

Note: Bold values indicate the best-performing configuration for each metric within the corresponding query rarity group.

For common-battle queries, the baseline LLM achieves relatively strong results, with an average score of 0.835 across the four metrics. The raw-text-augmented model achieves a similar average score of 0.868, with improvements of 7.6% in recall, 0.96% in precision, and 2.49% in exact match. The KB2 JSON-augmented model reaches an average score of 0.871, which is the highest among the three main model configurations, mainly due to achieving the

highest recall of 0.956. Compared with the baseline, the JSON-augmented model improves recall by approximately 12.8% and F1 by 6.2%, while precision is slightly lower and exact match remains unchanged. These results illustrate that context augmentation provides only a moderate advantage for common-battle queries.

A substantially different pattern was observed for rare-battle queries. Here, the baseline LLM performs considerably worse, achieving a precision of 0.609, recall of 0.471, and exact match of 0.313. In contrast, the KB2 JSON-augmented model outperforms the baseline LLM by an average of approximately 81.1% across all four evaluation metrics, achieving precision = 0.913, recall = 0.964, and exact match = 0.656. This corresponds to relative improvements of 49.7% in precision and 104.4% in recall. The raw-text-augmented model also achieves a substantial average relative improvement of approximately 67.7% over the baseline LLM across the four metrics, while the JSON-augmented configuration remains the strongest overall.

Overall, these results suggest that external context containing battle representations provides only a limited advantage for common-battle queries, where the baseline LLM already performs well. However, it becomes much more beneficial for rare-battle queries, where structured JSON battle representations considerably improve the model's ability to identify relevant battles, at least in the Oracle-context generation setting.

One representative example for JSON-augmented model advantage was observed for the query q089 *From the provided battle set, identify battles where armoured trains directly influenced the tactical outcome by providing decisive fire support, disabling an opposing train, or causing enemy troops to abandon a key position.*. As Table 5.9 shows, the baseline LLM failed to identify both golden battles, the Battle of Motovylivka and the Battle of Vapniarka. Instead it selected two battles belonging to closed-set negatives such as Battle of Antrea and the Battle of Vilppula. This resulted in scores 0.0 for all 4 metrics.

Table 5.9: Oracle-context generation results for the Motovylivka and Vapniarka battle query

Configuration	Selected battles	Gold battles found	Precision	Recall	F1	Exact Match
Baseline LLM	Antrea; Vilppula	0/2	0.000	0.000	0.000	No
JSON KB1	Motovylivka; Vapniarka	2/2	**1.000**	**1.000**	**1.000**	**Yes**
JSON KB2	Motovylivka; Vapniarka	2/2	**1.000**	**1.000**	**1.000**	**Yes**

This baseline-model failure is particularly interesting because it illustrates the problem of underrepresented knowledge about rare battles. The Battle of Motovylivka and the Battle of Vapniarka are relatively obscure engagements associated with the Ukrainian conflicts of 1918–1919. The Battle of Motovylivka was fought on 18 November 1918 between the Directorate of Ukraine and the Ukrainian State during the Ukrainian Civil War. Contemporary summaries, including our battle representation derived from the corresponding Wikipedia article, explicitly state that both sides employed armoured trains and that a Directorate armoured train disabled the opposing Hetman train (Wikipedia contributors, 2026a). The Battle of Vapniarka was fought during the Ukrainian–Soviet War in July 1919 around the strategically important Vapniarka railway station. Ukrainian forces used an improvised armoured train consisting of a cannon mounted on a railway platform, which contributed to panic and withdrawal among the Bolshevik forces (Wikipedia contributors, 2026b). Table 5.10

illustrates the corresponding evidence stored in our local battle representations, which directly supported the JSON-augmented LLM in identifying these gold battles.

Table 5.10: Evidence supporting the identification of gold-standard battles in JSON representations

Gold battle	Evidence available in JSON
Battle of Motovylivka	Directorate armoured train disabled the opposing Hetman armoured train with an artillery hit, forcing it to withdraw.
Battle of Vapniarka	Improvised armoured train with a cannon supported the attack and contributed to the Bolshevik withdrawal from Vapniarka.

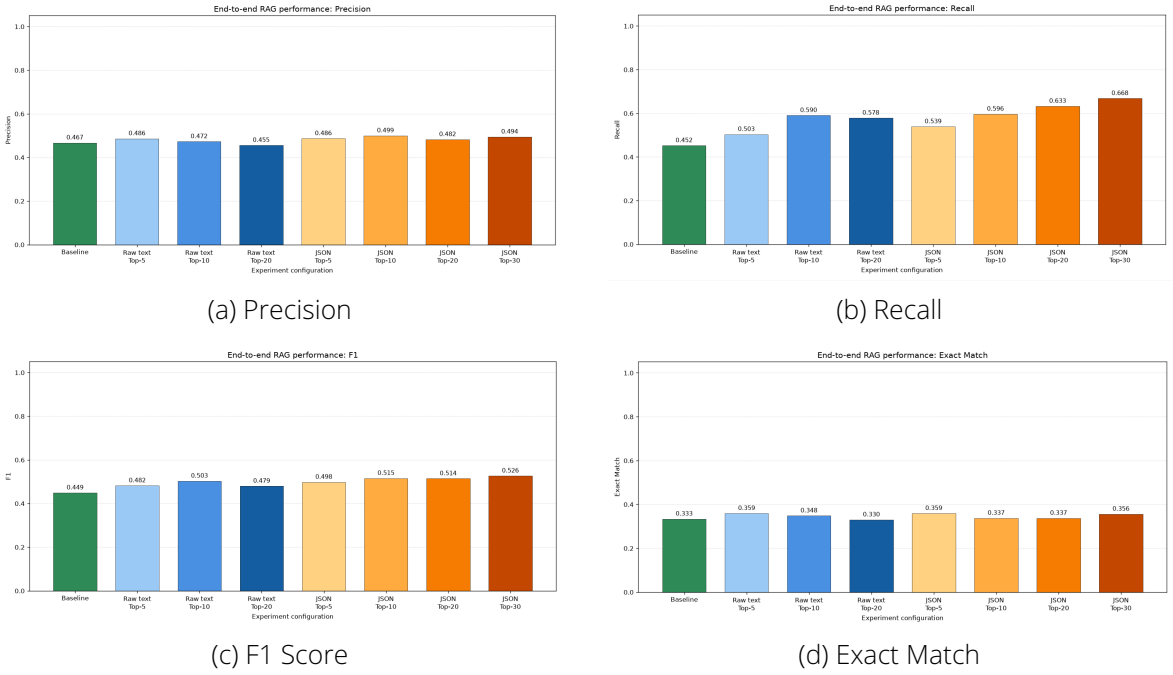
In this example, the baseline model still produced plausible explanations for other candidate battles, but it lacked sufficient knowledge to identify the correct engagements. In contrast, the JSON-augmented model selected both gold battles and achieved perfect scores across all four evaluation metrics.

5.3 End-to-End RAG Evaluation Results

As previously discussed in Chapter 4.4, the goal of the final experiment is to evaluate the complete RAG pipeline, including both the retrieval and generation steps, and to compare the augmented models with the baseline LLM. In this setting, a predefined candidate set of battles is not provided. The baseline LLM receives only the query, while the augmented LLMs receive the top-k battle instances retrieved by the hybrid retrieval pipeline. In the first sub-experiment, we evaluate all three model configurations across the complete query set, excluding the Similarity query type. Additionally, we test the JSON-augmented model using different numbers of retrieved battles, with $k = 5, 10, 20$, and 30, while the raw-text-augmented model is evaluated with $k = 5, 10$, and 20. In the two subsequent sub-experiments, we again evaluate the models separately for common- and rare-battle queries and across different query types. Unlike in the Oracle-context generation setting, we additionally apply faithfulness and context relevance metrics to evaluate the quality of the retrieved context.

5.3.1 Overall Performance and Error Analysis

The overall end-to-end RAG results across all 92 queries show that the JSON-augmented configuration performs similarly to the baseline LLM in terms of precision and exact-match accuracy, while achieving higher recall across the tested top-k settings. Table 5.11 and Figure 5.5 show the metric results for all three model configurations across different top-k settings. The highest precision for the JSON-augmented model was achieved at $k = 10$, where precision increased to 0.499 compared with 0.467 for the baseline LLM, corresponding to an improvement of approximately 7.0%. Compared with raw-text augmentation at the same context size, the JSON-augmented model achieved a 5.7% higher precision.



(a) Precision

(b) Recall

(c) F1 Score

(d) Exact Match

Figure 5.5: Overall performance comparison of baseline, raw-text-augmented, and JSON-augmented configurations in the end-to-end RAG experiment across Precision, Recall, F1, and Exact Match, using different numbers of retrieved battles (top-k).

Table 5.11: Overall end-to-end generation performance across the baseline, raw-text-augmented, and JSON-augmented configurations with different top-k context sizes

	Baseline	Raw-text			JSON			
Metric	Base	Top-5	Top-10	Top-20	Top-5	Top-10	Top-20	Top-30
Precision	0.467	0.486	0.472	0.455	0.486	**0.499**	0.482	0.494
Recall	0.452	0.503	0.590	0.578	0.539	0.596	0.633	**0.668**
F1	0.449	0.482	0.503	0.479	0.498	0.515	0.514	**0.526**
Exact Match	0.333	**0.359**	0.348	0.330	**0.359**	0.337	0.337	0.356

Note: Bold values indicate the best-performing configuration for each metric across all evaluated context sizes.

For recall, the advantage of the structured JSON context is more visible. At $k = 10$, the JSON-augmented model achieved a recall of 0.596, compared with 0.452 for the baseline LLM and 0.590 for the raw-text-augmented model, corresponding to improvements of approximately 31.9% and 1.0%, respectively. When the context size was increased to 20 retrieved battles, JSON recall further improved to 0.633, representing an increase of approximately 40.0% over the baseline LLM and 9.4% over the raw-text-augmented model at the same context size. At the same time, JSON@20 also achieved higher precision than both the baseline LLM (+3.2%) and raw-text@20 (+5.9%).

Overall, in the complete RAG setting, structured JSON battle representations primarily improve the model's ability to identify a larger proportion of relevant battles, while providing only modest gains in precision. Unlike in the Oracle-context experiment, the overall performance of the JSON-augmented model remains relatively close to the baseline LLM. This is likely because final answer quality in the end-to-end setting is additionally constrained by retrieval errors and by whether the relevant battles are present in the retrieved top-k context.

In the subsequent experiments, we chose a context size of $k = 20$, as it provided the best balance between recall and precision while also avoiding run errors for both the raw-text-augmented and JSON-augmented settings. Figure 5.6 illustrates the relationship between the number of retrieved JSON battle records and the number of generation errors. No errors were observed for context sizes from $k = 5$ to $k = 20$. However, starting from $k = 25$, generation failures began to appear and reached their highest number at $k = 40$. These failures mainly included response parsing errors and generations that exceeded the maximum token limit.

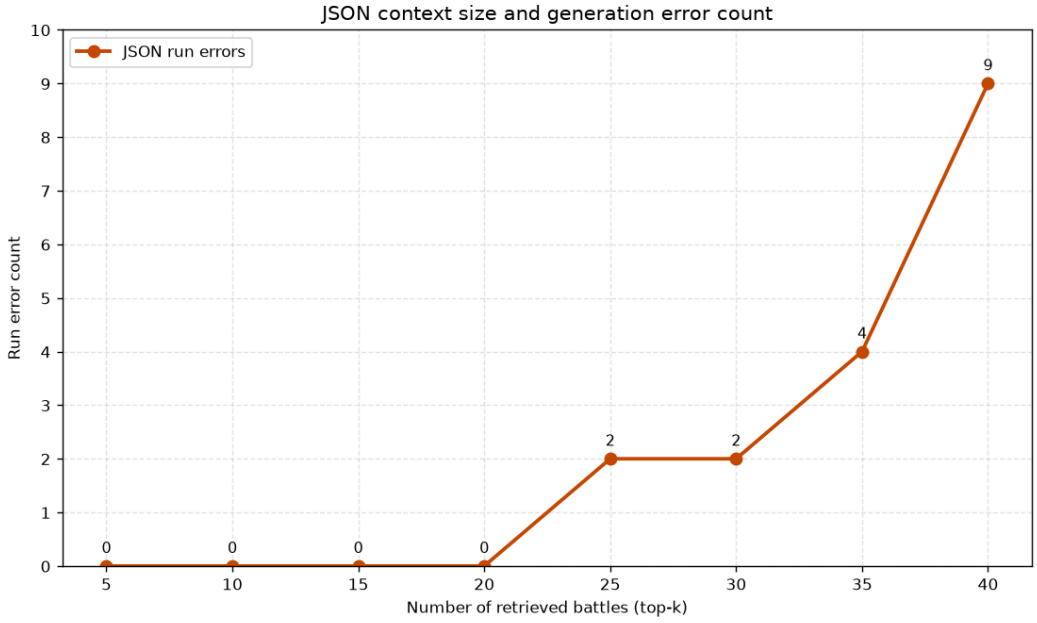


Figure 5.6: Number of generation run errors in the JSON-augmented end-to-end RAG experiment as a function of the number of retrieved battle representations (top-k).

5.3.2 Faithfulness and Context Relevance

Faithfulness and context relevance were evaluated separately on the complete query set of 92 queries, excluding the Similarity query type. Figure 5.7 presents the faithfulness scores and evaluation configuration. Macro faithfulness, where each query receives equal weight, reaches 0.896, while micro faithfulness, where each evaluated statement receives equal weight, reaches 0.904. This means that 90.4% of the evaluated statements were supported by the retrieved context. These results indicate that the JSON-augmented model largely relied on the retrieved context to generate factually grounded answers.

Context relevance was evaluated for $k = 5$, 10, and 20 to analyze how the amount of retrieved context affects the proportion of retrieved battle records that are actually relevant to answering the query. Table 5.12 presents the context relevance results and evaluation configuration. Figure 5.8 additionally visualizes the relationship between the number of retrieved battles and the average context relevance score. Particularly, it illustrates that as the number of retrieved battles increases, average context relevance steadily decreases, from 61.3% at $k = 5$ to 49.5% at $k = 20$. A context relevance score of 61% indicates that, on average, approximately 61% of the retrieved battle records contained information relevant to answering the corresponding query.

```
================================================================================
FAITHFULNESS EVALUATION FINISHED
================================================================================
Average faithfulness: 0.8957
Micro faithfulness: 0.9037
Evaluated queries: 92
Skipped queries: 0
Evaluation errors: 0
Report saved to: results\e2e_rag\top20_json_faithfulness_report.json
(hybridb_env) PS C:\Users\vlad8\Projects-VS-Code\HybridB>
```

Figure 5.7: Faithfulness evaluation results for the JSON-augmented end-to-end RAG configuration with top-20 retrieved battle representations.

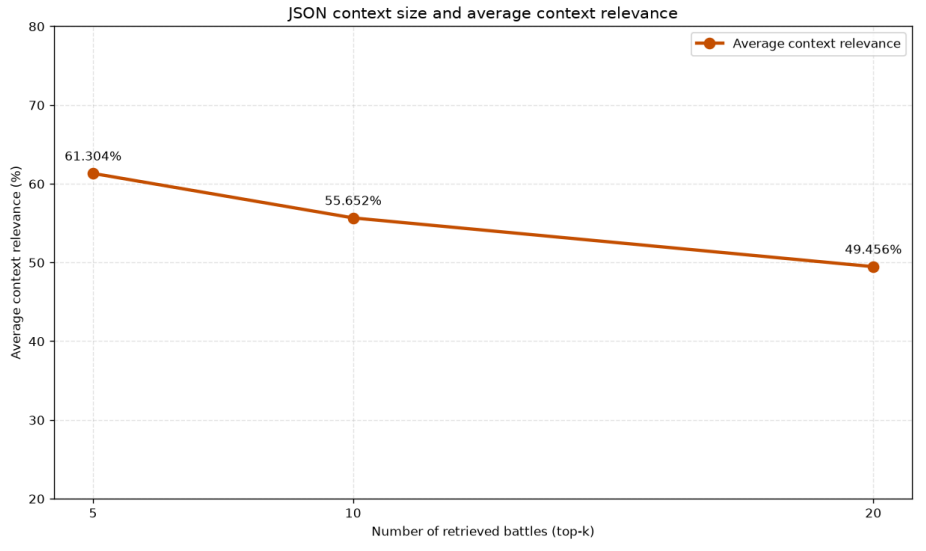


Figure 5.8: Average context relevance in the JSON-augmented end-to-end RAG experiment as a function of the number of retrieved battle representations (top-k).

Table 5.12: Context relevance results for different numbers of retrieved JSON battle records

Top-k	Average Context Relevance	Relevant Records	Total Context Records	Evaluation Errors
5	**61.304%**	282	460	0
10	55.652%	512	920	0
20	49.457%	910	1,840	0

Note: Bold values indicate the highest average context relevance score.

5.3.3 End-to-end RAG Performance for Common and Rare Battles

In the next sub-experiment, we assessed the performance of the complete pipeline separately for common- and rare-battle queries. We wanted to examine whether a similar trend would appear to the one observed in the Oracle-context generation experiment, where the augmented LLMs achieved substantial improvements over the baseline LLM for rare-battle queries. Table 5.13 represents the exact metric scores for the three model configurations across all four evaluation metrics.

Table 5.13: End-to-end RAG performance for common and rare battle queries across baseline, raw-text-augmented, and JSON-augmented configurations

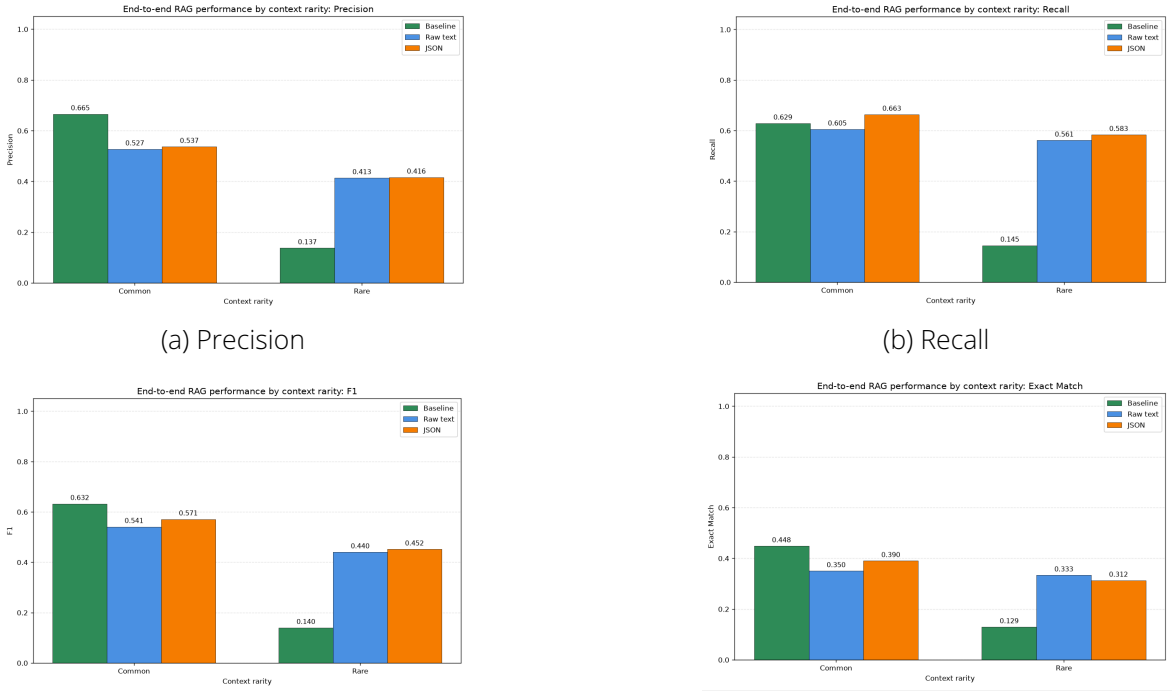
	Common			Rare		
Metric	Baseline	Raw text	JSON	Baseline	Raw text	JSON
Precision	**0.665**	0.527	0.537	0.137	0.413	**0.416**
Recall	0.629	0.605	**0.663**	0.145	0.561	**0.583**
F1	**0.632**	0.541	0.571	0.140	0.440	**0.452**
Exact Match	**0.448**	0.350	0.390	0.129	**0.333**	0.312

Note: Bold values indicate the best-performing configuration for each metric within the corresponding query rarity group.

As shown in Figure 5.9, for common-battle queries, the baseline LLM remains highly competitive and achieves the highest precision, F1, and exact-match scores, with values of 0.665, 0.632, and 0.448, respectively. The JSON-augmented model achieves the highest recall of 0.663, compared with 0.629 for the baseline LLM. However, its precision, F1, and exact-match scores remain lower than those of the baseline.

Similar to the Oracle-context generation experiment, we observed a substantially different pattern for rare-battle queries. Both context-augmented models again strongly outperform the baseline LLM across all four evaluation metrics. The JSON-augmented model improves precision from 0.137 to 0.416 (+203.6%), recall from 0.145 to 0.583 (+302.1%), F1 from 0.140 to 0.452 (+222.9%), and exact match from 0.129 to 0.312 (+141.9%). This corresponds to an average relative improvement of approximately 217.6% over the baseline. The raw-text-augmented model shows a similar pattern, improving precision to 0.413, recall to 0.561, F1 to 0.440, and exact match to 0.333, corresponding to an average relative improvement of approximately 215.2% over the baseline.

Overall, the results show that external context provides only a limited benefit for common-battle queries, while it becomes substantially more helpful for rare-battle queries.



(a) Precision

(b) Recall

(c) F1 Score

(d) Exact Match

Figure 5.9: Comparison of baseline, raw-text-augmented, and JSON-augmented configurations in the end-to-end RAG experiment across common and rare historical battle queries, evaluated using Precision, Recall, F1, and Exact Match.

5.3.4 End-to-end RAG Performance by Query Type

Next, we evaluate the complete pipeline separately for each query type to examine whether its performance varies across different query categories. All metric results are provided in Table 5.14, Table 5.15 and Table 5.16.

Table 5.14: End-to-end RAG performance across Temporal, Spatial/Terrain, and Participant-related query types

	Temporal			Spatial / Terrain			Participant-related		
Metric	Baseline	Raw text	JSON	Baseline	Raw text	JSON	Baseline	Raw text	JSON
Precision	**1.000**	0.833	0.823	0.218	0.340	**0.369**	0.572	**0.592**	0.562
Recall	0.887	0.887	**0.900**	0.205	0.449	**0.522**	0.520	**0.833**	0.828
F1	**0.917**	0.832	0.809	0.210	0.374	**0.417**	0.535	**0.641**	0.614
Exact Match	**0.800**	0.600	0.600	0.182	**0.231**	**0.231**	0.400	**0.429**	0.400

Note: Bold values indicate the best-performing configuration for each metric within the corresponding query type.

Table 5.15: End-to-end RAG performance across Narrative/Tactical, Outcome-related, and Weapon/Unit-related query types

	Narrative / Tactical			Outcome-related			Weapon / Unit-related		
Metric	Baseline	Raw text	JSON	Baseline	Raw text	JSON	Baseline	Raw text	JSON
Precision	0.295	**0.443**	0.341	**0.425**	0.358	0.393	0.483	**0.497**	0.437
Recall	0.309	**0.512**	0.469	0.425	0.394	**0.465**	0.463	**0.725**	0.613
F1	0.298	**0.456**	0.372	**0.424**	0.353	0.415	0.467	**0.565**	0.470
Exact Match	0.227	**0.400**	0.273	**0.333**	0.182	**0.333**	**0.350**	0.250	0.250

Note: Bold values indicate the best-performing configuration for each metric within the corresponding query type.

Table 5.16: End-to-end RAG performance for Similarity queries across baseline, raw-text-augmented, and JSON-augmented configurations

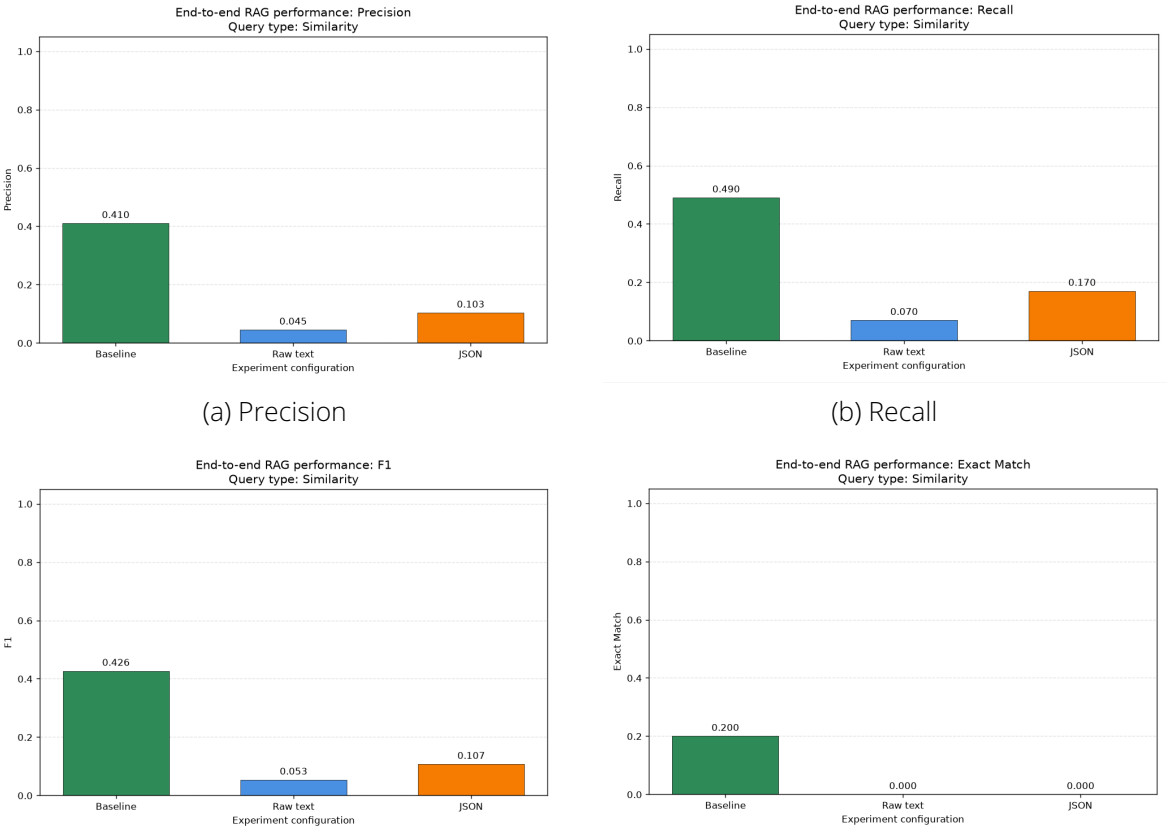
	Similarity		
Metric	Baseline	Raw text	JSON
Precision	**0.410**	0.045	0.103
Recall	**0.490**	0.070	0.170
F1	**0.426**	0.053	0.107
Exact Match	**0.200**	0.000	0.000

Note: Bold values indicate the best-performing configuration for each metric.

The query-type experiment shows that the effect of JSON augmentation varies considerably across different query categories. When the four evaluation metrics are averaged within each query type, the JSON-augmented model outperforms the baseline in four of the seven categories: Narrative/Tactical, Participant-related, Spatial/Terrain, and Weapon/Unit-related queries. The strongest improvement is observed for Spatial/Terrain queries, where the average score increases from 0.204 to 0.385, corresponding to a relative improvement of approximately 88.8%. Narrative/Tactical queries improve by 28.9% compared with the baseline model, while Participant-related queries show an 18.6% increase in the average score. Weapon/Unit-related queries, on the other hand, show only a minor improvement of 0.4%.

The main advantage of JSON augmentation is observed in the recall metric. Here, the JSON-augmented model achieves higher recall than the baseline in six of the seven query types. In particular, for Spatial/Terrain queries, the JSON-augmented model outperforms the baseline by 154.6%, increasing recall from 0.205 to 0.522. A similar trend is observed for Participant-related queries, with a 59.2% improvement over the baseline, followed by Narrative/Tactical queries with a 51.8% improvement and Weapon/Unit-related queries with an improvement of approximately 32%. These results indicate that structured context mainly improves the model's ability to recover relevant battles; however, this advantage is not universal. Similarity queries represent a clear failure case, where recall decreases from 0.490 for the baseline model to 0.170 for the JSON-augmented model. A similar pattern is observed for F1, where the baseline model substantially outperforms the JSON-augmented LLM. Temporal queries, as shown in Table 5.14, are also handled well by the baseline model, which achieves slightly better results than the augmented configurations.

Particularly interesting results are observed for the Similarity query type, as illustrated in Figure 5.10. The JSON-augmented model achieves substantially lower scores than the baseline LLM across all four evaluation metrics. Across precision, recall, and F1, the baseline LLM outperforms the JSON-augmented model by approximately 261% on average, while exact match is 0.0 for the JSON-augmented model. During the analysis, we found that this poor performance is primarily related to failures at the retrieval stage. In five of the ten Similarity queries, none of the corresponding gold battles appeared within the top 20 retrieved records, which prevented the JSON-augmented model from producing a correct gold answer.



(a) Precision

(b) Recall

(c) F1 Score

(d) Exact Match

Figure 5.10: Comparison of baseline, raw-text-augmented, and JSON-augmented configurations for Similarity queries in the end-to-end RAG experiment, evaluated using Precision, Recall, F1, and Exact Match.

A representative example is the first Similarity query: *Find battles similar to Cannae in which encirclement or converging attacks trapped a large enemy force and led to catastrophic defeat.* The gold set contains the Battle of Tannenberg, the Battle of Stalingrad, the Battle of Kyiv (1941), and the Battle of Białystok–Minsk. However, the hybrid retrieval pipeline failed to retrieve any of these battles within the top 20 results. As illustrated in Table 5.17, the hybrid retrieval algorithm instead returned another semantically similar set of battles associated with encirclement and catastrophic defeat, including the Battle of Changping, the Battle of Lake Trasimene, the Battle of Walaja, and the Falaise Pocket. However, none of these battles belong to the predefined gold set. Consequently, the JSON-augmented model could not recover any gold battle from the provided context, resulting in a recall of 0 for this query.

Table 5.17: Examples of retrieval failures for Similarity queries in the end-to-end RAG experiment

Query	Gold battles	Gold in top-20	Example retrieved alternatives	JSON Recall
q01 – Similar to Cannae	Tannenberg, Stalingrad, Kiev, Bialystok–Minsk	0/4	Changping, Trasimene, Walaja, Falaise Pocket	0.000
q08 – Similar to Stalingrad	Tannenberg	0/1	Kharkov, Korsun–Cherkassy, Demyansk, Kalach, Kiev, Vyazma	0.000

A similar pattern occurs for the q08 Similarity query, where the model was asked to find battles similar to Stalingrad in which an overextended attacking army was encircled by the enemy during a counteroffensive. The gold set contained only one battle, the Battle of Tannenberg, which was absent from the retrieved top-20 context. However, the hybrid retrieval pipeline still retrieved several plausible encirclement battles, including Korsun–Cherkassy, the Demyansk Pocket, Kalach, Kyiv, and the Vyazma Pocket.

These cases suggest an additional challenge in evaluating similarity-based queries. The previous query types were based on more explicit temporal, participant, spatial, weapon, or outcome constraints, whereas Similarity queries often involve several dimensions at once. As a result, different battles may be considered similar according to different properties. For example, q08 asks: *Find a battle similar to Stalingrad in which an attacking army became overextended and was ultimately encircled by a large counteroffensive.* Similarity to Stalingrad may refer to encirclement, engagement scale, army size, or the broader counteroffensive narrative. This makes Similarity queries particularly sensitive to incomplete gold sets, since a retrieved battle may be semantically plausible even if it is not included in the predefined gold set. Consequently, retrieval and generation quality may be underestimated for this query type. This limitation of the query design is discussed further in Chapter 6, Discussion.

Overall, the query-type experiment shows that the usefulness of structured augmentation depends strongly on the semantic characteristics of each query type.

5.4 JSON Knowledge Base Validation Results

In this section, we investigate the results of our JSON KB validation experiment discussed in Section 3.4. Table 5.18 presents the overall results as well as the results for each category. Overall, 73 of 80 sampled facts (91.25%) were supported by the corresponding Wikipedia source texts. No sampled fact was explicitly contradicted, while the remaining seven facts could not be sufficiently verified from the source. Five of these seven cases concerned the higher-level `outcome.result_type` field, suggesting that abstract outcome classifications are less directly grounded in the source wording than more factual fields such as participants, dates, and locations.

Table 5.18: Results of the JSON knowledge base validation experiment

Category	Evaluated	Supported	Not found	Support rate
Metadata	20	19	1	95.00%
Participants	20	20	0	100.00%
Outcome	20	15	5	75.00%
Military facets	20	19	1	95.00%
Overall	**80**	**73**	**7**	**91.25%**

6 Discussion

In this chapter, we first summarize the key findings and draw the main conclusions. After that, we discuss the main limitations of our implemented RAG pipelines and present several directions for future work.

6.1 Key Findings and Conclusions

6.1.1 Research Objective

The main objective of this thesis was to investigate whether structured representations of historical battles can improve retrieval and LLM-based question answering in this domain. To address this objective, we designed a structured JSON representation schema that combines factual, military, temporal, spatial, participant-related, outcome-related, and narrative information. We then constructed a large-scale knowledge base containing 11,561 historical battle representations and implemented lexical, dense, and hybrid retrieval pipelines. Finally, we evaluated how structured battle context affects answer generation compared with direct LLM querying and raw-text augmentation.

6.1.2 Answers to the Research Questions

We investigated three research questions. RQ1 focused on how historical battles can be represented through a structured schema that captures factual, military, participant-related, narrative, and other relevant information about the battle. RQ2 explored three different retrieval pipelines, lexical, dense, and hybrid, and compared their performance to determine which approach is most suitable for the complete end-to-end RAG pipeline. Finally, RQ3 investigated how structured battle context affects LLM-based question-answering performance compared with direct LLM querying and raw-text augmentation.

Regarding RQ1, we built a JSON battle representation schema that covers the mentioned aspects of a historical battle and can support different types of questions.

Regarding RQ2, we compared lexical, dense, and hybrid retrieval approaches for identifying relevant battle representations in our local knowledge base. The retrieval experiments showed that hybrid retrieval provided the most robust performance across the experiments,

although dense retrieval achieved higher results in several cases. Across our designed query set, hybrid retrieval outperformed lexical retrieval by around 21% on average across the six evaluation metrics and achieved slightly better overall performance than dense retrieval. The advantage of dense retrieval was particularly visible when relevant battle representations used different terminology from the query. Query type also had a noticeable effect on performance. Dense retrieval performed especially well for Temporal queries, while hybrid retrieval achieved the highest scores for Spatial/Terrain and Weapon/Unit-related queries. Overall, hybrid retrieval consistently improved over the lexical algorithm by combining both lexical and semantic relevance signals. However, its advantage over dense retrieval depended more strongly on the characteristics and formulation of the query.

Regarding RQ3, we evaluated structured battle representations in both Oracle-context and end-to-end RAG settings. In the Oracle-context experiment, the JSON-augmented model clearly outperformed both the baseline LLM and raw-text augmentation, with an average improvement of approximately 17.9% over the baseline and 6.8% over raw text, while the advantage was especially strong for rare-battle queries. In the complete RAG setting, the improvement was more moderate because final performance also depended on retrieval quality. The JSON-augmented model mainly improved recall and again showed the strongest gains for rare battles, where it outperformed the baseline across all four metrics. Overall, structured battle representations improve LLM-based question answering, particularly for rare or underrepresented battles, but their effectiveness in the complete RAG pipeline remains strongly dependent on whether relevant battles are successfully retrieved.

6.1.3 Overall Conclusion

Overall, this thesis demonstrated that historical battles can be represented through a structured schema and organized into a large-scale knowledge base containing 11,561 battle instances. We evaluated how these structured representations affect both retrieval and answer-generation quality under different experimental settings. Across our benchmark of 102 queries, the structured battle knowledge base generally improved the ability of augmented LLMs to identify relevant battles compared with the baseline model, with particularly strong benefits observed for rare-battle queries. We also compared three retrieval strategies and found that hybrid retrieval, which combines BM25 and dense rankings, provides the most robust overall performance in many cases. This makes it especially useful for queries that require both exact lexical matching and semantic similarity. At the same time, the experiments revealed several limitations related to retrieval quality, benchmark construction, schema normalization, and knowledge base coverage, which provide clear directions for future work.

6.2 Limitations and Future Work

6.2.1 Benchmark Grounding and Historical Validity

This limitation concerns the potential difference between the source used to construct the benchmark, which in our case is primarily Wikipedia, and the knowledge available to the evaluated baseline LLM. The queries, gold battle sets, and structured battle representations were constructed mainly from raw-text Wikipedia articles. Therefore, the augmented models receive evidence originating from the same source on which the queries and gold sets were based. In contrast, the baseline LLM relies on parametric knowledge acquired during training.

The exact composition of this training knowledge is unknown and cannot be reproduced in our experiments. Its knowledge may originate from different encyclopedias, websites, historical books, and other sources in which historical events may be described or interpreted differently from Wikipedia.

This issue is particularly relevant in the historical domain, where causal relationships, participant roles, and interpretations of decisive events may differ across sources. As a result, the baseline LLM may generate a historically plausible answer and identify battles that are relevant according to its internal knowledge, while still deviating from the predefined gold set. Such disagreement does not necessarily mean that the baseline answer is historically incorrect, but may instead reflect differences between historical sources and interpretations. Therefore, the reported evaluation results should primarily be interpreted as agreement with a Wikipedia-grounded benchmark rather than as a measure of absolute historical correctness.

A possible direction for future work would be to construct the local knowledge base, queries, and gold battle sets using multiple historical sources rather than relying primarily on Wikipedia. The resulting benchmark could then be manually reviewed by domain experts to verify the selected battles and reduce the influence of source-specific interpretations.

6.2.2 Wikipedia Data Quality

For our implementation, we use Wikipedia and Wikidata as a single source of truth for knowledge base construction, query and gold-set construction, and model augmentation with this knowledge. We consider this a limitation. The amount and quality of information available on Wikipedia vary considerably depending on the battle or historical event. Several obscure battles may contain only limited descriptions of tactics, narrative and event sequence, participants, and their roles. Besides, Wikipedia is collaboratively edited, and its content may be insufficiently verified, incomplete, or subject to later revision. As a result, our constructed battle representations may inherit inconsistencies and data gaps from Wikipedia.

6.2.3 Limited Knowledge Base Coverage

The next limitation is the insufficient coverage of our knowledge base. As previously discussed in the Methodology chapter, all battle instances extracted into our local knowledge base were associated with the Wikidata class battle (`Q178561`) and were retrieved using a SPARQL query that selected battles with a corresponding English Wikipedia page. This additional filter already reduced the number of battle instances from 18,655 to 11,561. Besides, Wikidata contains several additional military event classes that may also be useful to include in our knowledge base. As illustrated in Table 6.1, these may include siege, military operation, armed conflict, or military campaign. Additionally, Wikidata contains subclasses of battle that may represent more specific battle types, such as naval or air battles. Therefore, future work could investigate additional Wikidata classes that may be suitable for inclusion in our knowledge base and incorporate the corresponding instances.

Table 6.1: Potential Wikidata classes for extending the historical battle knowledge base

Wikidata class	Wikidata ID	Approx. instances	Relevance for extension
Battle	Q178561	18,655	Currently used
Siege	Q188055	3,918	High
Military operation	Q645883	3,093	High
Armed conflict	Q350604	530	Medium
Military campaign	Q831663	1,276	Medium

6.2.4 Field-Aware Lexical Retrieval

Our implemented lexical retrieval pipeline does not explicitly consider the internal structure of the JSON battle representations, including field names and values. Instead, it treats each document as a single flat sequence of tokens.

This can be problematic when the lexical retrieval procedure returns syntactically similar but semantically different battle records. For instance, Table 6.2 shows that two records may contain the same keywords that also appear in the query *Find battles where Napoleon defeated the Prussian Army*, while still expressing opposite relationships between the participants. The wording is syntactically similar, but the semantic direction of who defeated whom is different. In this case, BM25 may rank both records highly even though only one of them is actually relevant to the query.

Table 6.2: Example illustrating the importance of semantic relationships in battle retrieval

Query / document	Text
Query	*Find battles where Napoleon defeated the Prussian Army.*
Relevant record	*Napoleon's French army defeated the Prussian army in the battle.*
Non-relevant record	*The Prussian army defeated Napoleon's French army in the battle.*

One potential extension to our pipeline would be to add query-dependent metadata filtering that analyzes the query formulation and makes lexical retrieval operate only on the relevant JSON fields. For example, a query asking about terrain could prioritize fields such as terrain and location, while queries asking about participant tactics could give more attention to the `narrative_patterns` and `tactics` fields.

This solution could be further combined with field-aware BM25 scoring. Previous work on structured lexical retrieval has extended the BM25 algorithm by assigning different weights to different document fields. For example, this work proposed BM25E, building on BM25F, where term frequencies from different structural fields are weighted before being used by the BM25 scoring function (Lu et al., 2006). We could adapt a similar approach and assign different weights to different JSON fields depending on the retrieval task.

One question that still remains is how the retrieval algorithm can recognize which query patterns it should look for when the query is expressed in natural language. Queries can be quite complex, and it may be difficult to identify which fields should be prioritized. For

example, query q050 asks: *Find a battle where elephants attacked first, infantry then became locked in combat, and returning cavalry finally struck the rear of the losing army.* Here, the lexical retrieval algorithm would need to consider participant information, event order, and unit types. Recognizing these requirements automatically may be difficult.

One potential solution would be to use an LLM to identify the relevant fields that the retrieval algorithm should then search. Another solution could be to use database-like queries in which the user explicitly refers to the relevant field names. For query q050, an example of such a formulation is shown in Listing 6.1. However, matching field names alone would still not be sufficient, since the field values also need to be normalized and comparable. This leads to the next limitation concerning field normalization.

```
SELECT battle
FROM battles
WHERE weapons_or_units CONTAINS 'war elephants'
  AND tactics CONTAINS 'cavalry flanking'
  AND event_sequence MATCHES (
      'elephant attack'
      THEN 'infantry engagement'
      THEN 'cavalry return'
      THEN 'rear attack'
  )
  AND cavalry_target = 'losing army'
  AND outcome = 'decisive defeat';
```

Listing 6.1: SQL-style representation of a complex historical battle query involving war elephants, cavalry flanking, and a specific sequence of tactical events.

6.2.5 Schema Normalization

This limitation concerns the degree of normalization in our battle representation schema. Initially, we provided an empty JSON schema to the LLM filler model, but the model was not instructed to use predefined values for all fields. Instead, the model independently interprets each battle passage and may express semantically similar information using different terms. For example, outcome-related fields may contain values such as `decisive victory`, `successful defense`, `successful withdrawal`, or `defeat`, while the LLM may assign alternative formulations describing the same meaning. We consider this a limitation because it reduces consistency across battle representations and makes keyword filtering or rule-based queries more difficult. Lexical retrieval may also become less accurate because semantically similar battle aspects can be described using different terms, as shown in Table 6.3.

Table 6.3: Examples of semantic differences between query terms and alternative descriptions in JSON representations

Query term	Alternative description in JSON
Cavalry flanking maneuver	Mounted troops attacked from the side
Decisive victory	Overwhelming defeat of the opposing army
Retreat	Withdrawal from the battlefield
Ambush	Surprise attack from a concealed position
Defensive formation	Troops established a protective line
River crossing	Army advanced across a waterway
Encirclement	Enemy forces were surrounded
Reinforcements arrived	Additional troops joined the battle

Future versions of the representation schema could introduce a predefined set of labels that the LLM is instructed to use during the filling procedure, while still allowing free-text values where necessary. This could further improve consistency across battle representations and, consequently, retrieval performance.

6.2.6 Query Set Construction

The construction of the query set also introduces several limitations. First, it would be preferable for the generated queries and the corresponding gold battle sets to be manually verified against reliable historical sources to ensure that all relevant battles are included in the gold set and that the questions are formulated correctly, reducing the risk of potential misinterpretation. Another limitation concerns the batch-wise query construction procedure. Queries were generated using GPT-5.6 Sol from subsets of approximately 1,000 battle instances at a time. This means that the model could only consider potential gold battles available within the current batch and could miss relevant battles from other batches across the complete set of 11,561 prepared instances. This limitation is especially important for Similarity queries, where identifying similar battles may require awareness of the complete battle set. Therefore, semantically reasonable answers produced by the baseline model during our experiments may still be counted as false positives when they include relevant battles that fall outside the predefined gold set. Poor performance on Similarity queries should therefore be interpreted as a combination of retrieval failures and potentially incomplete gold labels, rather than purely as a generation failure of the augmented models.

Bibliography

Anthropic. 2026a. Intro to Claude. `https://platform.claude.com/docs/en/intro`. Accessed: 2026-10-26.

Anthropic. 2026b. Introducing Claude Sonnet 5.5. `https://www.anthropic.com/claude-sonnet-5-5`. Accessed: 2026-10-26.

Dorian Brown. 2021. Rank-BM25: A Collection of BM25 Algorithms in Python. Python library. Accessed: 2026-10-26.

Nathanael Chambers and Dan Jurafsky. 2010. A Database of Narrative Schemas. In *Proceedings of the Seventh International Conference on Language Resources and Evaluation (LREC'10)*, Valletta, Malta. European Language Resources Association (ELRA).

Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. 2009. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759. ACM.

Rebecca Coulson, Manuel Vindiola, Song Park, and Kevin Corder. 2023. Action clusters: an NLP-powered approach to detecting similarities of actions within battles. In *Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications V*, volume 12538.

Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2024. The Faiss Library. *arXiv preprint arXiv:2401.08281*.

Shahul Es, Jithin James, Luis Espinosa Anke, and Steven Schockaert. 2024. RAGAs: Automated Evaluation of Retrieval Augmented Generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, pages 150–158, St. Julians, Malta. Association for Computational Linguistics.

Gemini Team. 2023. Gemini: A Family of Highly Capable Multimodal Models.

Google. 2026. Gemini 3.6 Flash. `https://ai.google.dev/gemini-api/docs/models/gemini-3.6-flash`. Accessed: 2026-10-26.

Saiping Guan, Xueqi Cheng, Long Bai, Fujun Zhang, Zixuan Li, Yutao Zeng, Xiaolong Jin, and Jiafeng Guo. 2023. What is Event Knowledge Graph: A Survey. *IEEE Transactions on Knowledge and Data Engineering*, 35(7):7569–7589.

Sil Hamilton, Matthew Wilkens, and Andrew Piper. 2026. NarraBench: A Comprehensive Framework for Narrative Benchmarking. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3786–3801, Rabat, Morocco. Association for Computational Linguistics.

Steve Harris and Andy Seaborne. 2013. SPARQL 1.1 Query Language. W3c recommendation, World Wide Web Consortium (W3C).

Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, Online. Association for Computational Linguistics.

Jinqi Lai, Wensheng Gan, Jiayang Wu, Zhenlian Qi, and Philip S. Yu. 2024. Large Language Models in Law: A Survey. *AI Open*, 5:181–196.

Viet Dac Lai. 2022. Event Extraction: A Survey. *Preprint*, arXiv:2210.03419.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474.

Sheng Liang, Hang Lv, Zhihao Wen, Yaxiong Wu, Yongyue Zhang, Hao Wang, and Yong Liu. 2025. Adaptive Schema-aware Event Extraction with Retrieval-Augmented Generation. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 7927–7946, Suzhou, China. Association for Computational Linguistics.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173.

Wei Lu, Stephen E. Robertson, and Andrew Macfarlane. 2006. Field-Weighted XML Retrieval Based on BM25. In *Advances in XML Information Retrieval and Evaluation*, volume 3977 of *Lecture Notes in Computer Science*, pages 161–171. Springer.

Yujie Luo, Xiangyuan Ru, Kangwei Liu, Lin Yuan, Mengshu Sun, Ningyu Zhang, Lei Liang, Zhiqiang Zhang, Jun Zhou, Lanning Wei, Da Zheng, Haofen Wang, and Huajun Chen. 2025. OneKE: A Dockerized Schema-Guided LLM Agent-based Knowledge Extraction System. In *Companion Proceedings of the ACM on Web Conference 2025*, pages 2871–2874. Association for Computing Machinery.

Martin Majlis. Wikipedia-API: Python API for Wikipedia. Python Package Index (PyPI). Accessed: 2026-10-26.

OpenAI. 2025. gpt-oss-120b & gpt-oss-20b Model Card. OpenAI. Published: August 5, 2025. Accessed: 2026-10-26.

Stephen E. Robertson and Hugo Zaragoza. 2009. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389.

Bhaskarjit Sarmah, Benika Hall, Rohan Rao, Sunil Patel, Stefano Pasquali, and Dhagash Mehta. 2024. HybridRAG: Integrating Knowledge Graphs and Vector Retrieval Augmented Generation for Efficient Information Extraction. In *Proceedings of the 5th ACM International Conference on AI in Finance*, pages 608–616. Association for Computing Machinery.

ScaDS.AI Dresden/Leipzig. Center for Scalable Data Analytics and Artificial Intelligence. Official website. Accessed: 2026-10-26.

Siqi Shen and Amanmeet Garg. 2025. Adapting Large Language Models for Movie Domain with Narrative Understanding Tasks. In *Proceedings of the 29th Conference on Computational Natural Language Learning*, pages 187–200.

Denny Vrandečić and Markus Krötzsch. 2014. Wikidata: A Free Collaborative Knowledgebase. *Communications of the ACM*, 57(10):78–85.

Dandan Wang and Shiqing Zhang. 2024. Large language models in medical and healthcare fields: applications, advances, and challenges. *Artificial Intelligence Review*, 57(11):299.

Wikidata contributors. 2026. Wikidata: Introduction. Accessed: 2026-10-26.

Wikipedia contributors. 2026a. Battle of Motovilivka. Wikipedia, The Free Encyclopedia. Accessed: 2026-10-26.

Wikipedia contributors. 2026b. Battle of Vapniarka. Wikipedia, The Free Encyclopedia. Accessed: 2026-10-26.

Zairun Yang, Yilin Wang, Zhengyan Shi, Yuan Yao, Lei Liang, Keyan Ding, Emine Yilmaz, Huajun Chen, and Qiang Zhang. 2025. EventRAG: Enhancing LLM Generation with Event Knowledge Graphs. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16967–16979.

Yu Zhang, Xiusi Chen, Bowen Jin, Sheng Wang, Shuiwang Ji, Wei Wang, and Jiawei Han. 2024. A Comprehensive Survey of Scientific Large Language Models and Their Applications in Scientific Discovery. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 8783–8817.